

Aufgabe 4 (Java)

(12 Punkte)

Vervollständigen Sie folgenden Java-Code eines einfachen Web-Servers, der zum gleichzeitigen Bearbeiten von Anfragen **NTHREADS** Aktivitäten verwendet und höchstens **CAPACITY** unbearbeitete Anfragen zwischenspeichert.

- Ergänzen Sie in Feld 1 eine für diesen Web-Server angemessene Warteschlange.
- Ergänzen Sie in Feld 2 eine angemessene Überlaufstrategie (**Policy**).
- Ergänzen Sie in den Feldern 3 und 4 das Code-Gerüst zu einem funktionsfähigen parallelarbeitenden Web-Server.

```
import java.net.*; import java.util.concurrent.*;
class WebServer {
    private static final int NTHREADS = 100;
    private static final int CAPACITY = 100;
    private static final Executor exec = new ThreadPoolExecutor(
        NTHREADS, NTHREADS,
        0L, TimeUnit.MILLISECONDS,
1   [ ] ,
2   [ ]
    );
    public static void main(String[] args) throws Exception {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
3   [ ]
            handleRequest(connection);
4   [ ]
        }
    }
}
```

Aufgabe 3 (Parallele Minimumsreduktion)

(12 Punkte)

Die Methode `minTurnier` implementiert eine parallele Minimumsreduktion mithilfe des vollständigen Turnierverfahrens.

- a) Wenden Sie das Turnierverfahren auf folgendes Feld der Länge `len = 8` an, so dass am Ende das kleinste Element an erster Position im Feld steht.

9	7	1	0	3	2	5	8
---	---	---	---	---	---	---	---

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

- b) Wie viele Aktivitäten werden beim Turnierverfahren für ein Feld der Länge `len` benötigt?

- c) Nach wie vielen Schritten sind alle Aktivitäten bei einem Feld der Länge `len` fertig?

- d) Vervollständigen Sie die folgende Implementierung der parallelen Minimumsreduktion (ohne Ausnahmebehandlung). Erzeugen Sie dazu in Feld 1 eine ausreichend große Barriere. Realisieren Sie die Schrittweite und damit die Terminierung der inneren Schleife in Feld 2. Fügen Sie in den Feldern 3 und 4 nur falls nötig entsprechende Synchronisationsanweisungen ein. Sorgen Sie in Feld 5 dafür, dass das Ergebnis erst zurückgegeben wird, nachdem die parallel laufenden Aktivitäten die Berechnung abgeschlossen haben.

```

import java.util.concurrent.*;

public class Turnier {

    public static int minimum(final int[] feld) throws Exception{
        final int len = feld.length;
        Thread t = null;

        final CyclicBarrier b = new CyclicBarrier( 1);
        for (int i = 0; i < len - 1; i += 2) {
            final int a = i;
            t = new Thread() {
                public void run() { try {
                    for (int s = 1; s < len;  2) { 3
 3

                    if ((a / 2) % s == 0 && a + s < len) {
                        feld[a] = Math.min(feld[a + s], feld[a]);
                    }
 4

                }
            } catch (Exception e) { } }
        };
        t.start();
 5

        return feld[0];
    }
}

```

Aufgabe 4 (Schreibtischlauf)

(10 Punkte)

Gegeben ist folgendes paralleles Java-Programm:

```
public class Output {
    static int a = 1;
    static int b = 2;
    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread() {
            public void run() {
                a = 4; b = 5;
                String text = a + " < " + b;
                System.out.println(text);
            }
        };
        Thread t2 = new Thread() {
            public void run() {
                String text = b + " < " + a;
                System.out.println(text);
            }
        };
        t1.start(); t2.start(); t1.join(); t2.join();
    }
}
```

- a) Welche verschiedenen Ziffern sind in der Ausgabe als erste Ziffer möglich?
- b) Welche verschiedenen Ziffern sind in der Ausgabe als letzte Ziffer möglich?
- c) Welche verschiedenen, unwahren Ungleichungen könnte das Programm ausgeben?
- d) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden?
- e) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden, wenn die Variablen **a** und **b** als **volatile** deklariert werden?
- f) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden, wenn die beiden **run**-Methoden als **synchronized** deklariert werden?

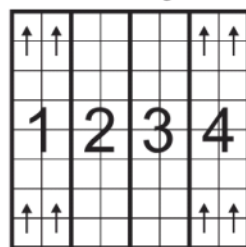
Aufgabe 3 (Geometrische Dekomposition)

(12 Punkte)

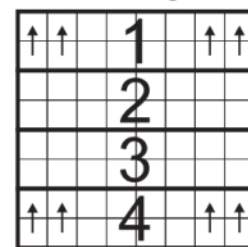
Beim Abspann eines Filmes wandert der Text von unten nach oben über den Bildschirm. Die notwendige Veränderung des Bildspeichers lässt sich mittels geometrischer Dekomposition parallelisieren, indem verschiedene Bereiche des Bildes von nebenläufigen Aktivitäten verschoben werden.

- a) In folgenden Bildern sind Ihnen zwei mögliche Aufteilungen eines 8x8 Pixel großen Bildschirms (inklusive Schieberichtung) auf 4 Aktivitäten gegeben. Nennen Sie je Aufteilung einen entscheidenden Vor- bzw. Nachteil, der die Parallelisierung des Filmabspanss beeinträchtigt.

Verteilung A

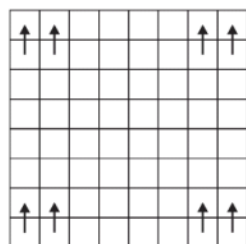


Verteilung B

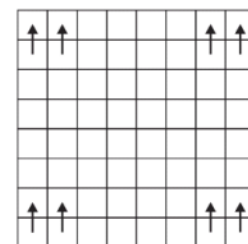


- b) Zeichnen Sie nach obigem Muster zwei weitere sinnvolle und möglichst effiziente Aufteilungen auf 4 Aktivitäten in folgende Diagramme ein.

Verteilung C



Verteilung D



- c) Vervollständigen Sie die folgende Implementierung des parallelen Filmabspanss (ohne Ausnahmebehandlung), sodass eine Verteilung nach dem Muster **A** auf `tCount` Aktivitäten realisiert wird. Erzeugen Sie dazu eine ausreichend große Barriere und fügen Sie in die mit * bezeichneten Platzhalter (nur falls nötig) entsprechende Synchronisationsanweisungen ein. Initialisieren und realisieren Sie in den mit + bezeichneten Platzhaltern die geometrische Dekomposition des Feldes.

Hinweis: Beachten Sie, dass die Bildschirmbreite nicht notwendigerweise ein Vielfaches der Zahl der Aktivitäten `tCount` ist.

```

import static java.lang.Math.*;
import java.util.concurrent.*;
public class Slide {
    public static void show(final int [][] screen) { /* Seiteneffekt */ }
    public static int [][] slideBlock(final int [][] screen,
        final int tCount, final int steps) throws Exception {
        show(screen); Thread t = null;
        final int maxX = screen.length;
        final int maxY = screen[0].length;

        final CyclicBarrier b = new CyclicBarrier(  );
 *
 +

        for (int i = 0; i < tCount; i++) {
            final int id = i;
            t = new Thread() {
                public void run() { try {
                    for (int j = 0; j < steps; j++) {
 *


                    for (int x =
 +


                    ) {
                        final int first = screen[x][0];
                        for (int y = 1; y < maxY; y++)
                            { screen[x][y - 1] = screen[x][y]; }
                        screen[x][maxY - 1] = first;
                    }
 *


                    if (id == 0) { show(screen); }
 *


                }
            } catch (Exception e) {} }
        }; t.start();
    }
    t.join(); return screen;
}
}

```

(13 Punkte)

a) Beschreiben Sie kurz die Idee des Monte Carlo Verfahrens zur Berechnung von π .

b) Warum eignet sich das Verfahren gut zur Parallelisierung? Begründen Sie ihre Antwort kurz.

c) Vervollständigen Sie die folgende Implementierung des parallelen Monte Carlo Verfahrens (ohne Ausnahmebehandlung), das `task` Prozesse verwendet um `count` Punkte zu platzieren. Erzeugen/Zerstören Sie dazu in den beiden mit 1 markierten Feldern einen passenden `ExecutorService` und einen entsprechenden `CompletionService`. Implementieren Sie in Feld 2 die eigentliche Simulation. Realisieren Sie in den zwei mit 3 markierten Feldern die Verteilung und Bearbeitung in einzelnen Tasks und in Feld 4 die abschließende Zusammenführung der Berechnungen.

Tipp: Verwenden Sie eine anonyme innere Klasse.

```

import java.util.Random;
import java.util.concurrent.*;
public class MonteCarlo {
    public static double pi(int tsk, int cnt) throws Exception {
        final ExecutorService exec =
             1

        final CompletionService<Integer> comp =
            new ExecutorCompletionService<Integer>( 1);
        final int pointsPerTask = cnt / tsk;
        for (int i = 0; i < tsk; i++) {
             3

            Random r = new Random();
            int sumpart = 0;
            for (int s = 0; s < pointsPerTask; s++) {
                double x = r.nextDouble();
                double y = r.nextDouble();

                 2
            }
             3
        }
        int sum = 0;
        for (int i = 0; i < tsk; i++) {
             4
        }
         1

        return (4. * sum) / cnt;
    }
}

```

Aufgabe 3 (Rang einer Liste)

(12 Punkte)

Die parallele Methode `rang` bestimmt den Rang ($1 + \text{Abstand zum Ende der Liste}$) jedes Elements in einer als Array repräsentierten, einfach verketteten Liste. Der Listenkopf ist das Element am Index 0. Jeder Eintrag ist der Index des Nachfolgers in der Liste. Das Listenende wird durch -1 markiert. Es werden aus Einfachheit keine Nutzdaten gespeichert.

- a) Bestimmen Sie den Rang jedes Elements in folgender Liste.

Liste: { 5 , 8 , 4 , 1 , 6 , 3 , -1 , 2 , 7 }

Rang: { , , , , , , 1 , , }

- b) Welchen Zeitaufwand hat die Rangbestimmung bei einer Liste der Länge n , wenn man nur eine Berechnungseinheit zur Verfügung hat?

Zeit $\in O(\text{$)

- c) Auf welchen Zeitaufwand kann man die Rangbestimmung bei einer Liste der Länge n optimieren, wenn man beliebig viele Berechnungseinheiten zur Verfügung hat? Welchen Arbeitsaufwand hat diese Variante?

Zeit $\in O(\text{$) Arbeit $\in O(\text{$)

Wie nennt sich dieses parallele Verfahren?

- d) Vervollständigen Sie die folgende Implementierung der parallelen Rangbestimmung (ohne Ausnahmebehandlung), die auf einer Liste mit `count` Elementen und ebensovielen Aktivitäten arbeitet. Sorgen Sie in dem mit 1 markierten Feld dafür, dass jede Aktivität genau soviel Arbeit macht, wie nötig ist. Realisieren Sie in den mit 2 markierten Feldern die notwendige Synchronisation für einen fehlerfreien Ablauf.

```

import java.util.concurrent.*;
public class Rang {
    private static int[] orig, copy;
    public static int[] rang(final int[] list) throws Exception {
        final int count = list.length;
        final int[] rang = new int[count];
        orig = list.clone(); copy = new int[count];
        final CyclicBarrier b = new CyclicBarrier(count,
            new Runnable() { public void run() {
                // swap orig/copy as single action after each await
                int[] tmp = orig; orig = copy; copy = tmp;
            }});
        Thread last = null;
        for (int i = 0; i < count; i++) { final int pos = i;
            last = new Thread() {
                public void run() { try {
                     2

                    // initialize rang and clone original data
                    if (orig[pos] == -1) { rang[pos] = 1; }
                    copy[pos] = orig[pos];

                     2

                    for (int step = 1;  1) { 2
                        

                        // update rang and list data
                        final int next = orig[pos];
                        if (next >= 0 && orig[next] == -1) {
                            rang[pos] = step + rang[next];
                        }
                        copy[pos] = next < 0 ? -1 : orig[next];

                         2
                    }

                     2

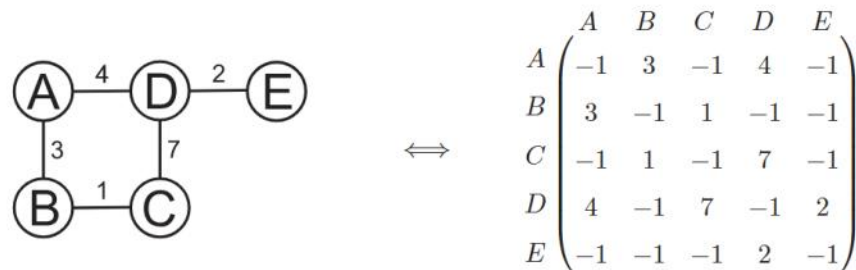
                } catch (Exception e) { e.printStackTrace();}}
            }; last.start();
        }
        last.join();
        return rang;
    }
}

```

Aufgabe 3 (Dijkstra/MapReduce)

(12 Punkte)

Der sequentielle Algorithmus von Dijkstra berechnet die kürzesten (leichtesten) Pfade in einem kantengewichteten Graphen von einem Startknoten zu einem/allen anderen Knoten im Graph. Im Folgenden betrachten wir eine mittels MapReduce parallelisierte Variante zur Berechnung aller kürzesten Pfade in einem ungerichteten Graphen. Ein Graph wird hier in Form einer Adjazenzmatrix gespeichert (siehe Beispiel). Gehen Sie zur Vereinfachung davon aus, dass die Adjazenzmatrix als globale Datenstruktur für die Map- und die Reduce-Phase zur Verfügung steht.



- a) Was ist die entscheidende Grundidee des MapReduce-Verfahrens?
- b) In der MapReduce-Variante des Dijkstra-Algorithmus wird MapReduce iterativ verwendet. Der Graph wird vom Startknoten aus in Breitensuche traversiert.
- Beschreiben Sie kurz in Worten was ein Schritt des Algorithmus berechnet.

In der Map-Phase (für einen Knoten X):

In der Reduce-Phase:

- Will man die kürzesten Wege vom Knoten A aus ermitteln, wird die map-Funktion in der ersten Iteration (nur) auf das Schlüssel-/Wertpaar (A,0) aufgerufen. Welche Ergebnisse liefert die Map-Phase dieser ersten Iteration?

- Wie oft und mit welchen Werten taucht der Knoten C nach der zweiten Iteration auf, ehe die Reduce-Phase ausgeführt wird?

- c) Implementieren Sie in folgendem Code-Gerüst eine Map-Funktion für den Dijkstra-Algorithmus. Beachten Sie hierbei, dass die Map-Funktion für genau einen Knoten aufgerufen wird und sie ihre verschiedenen Ergebnisse nacheinander mit der Funktion `void IntStream.put(int key, int value)` in den Ausgabe-Strom schreibt.

```
final static int NO_EDGE = -1;
int [][] graph; // adjacency matrix [source][destination]

public void map(int node, int distance, IntStream out) {
    final int nodeCount = graph.length;
```

- d) Mit welcher (semantikerhaltenden) Änderung der Adjazenzmatrix-Repräsentation ließe sich der Code für die Map-Funktion vereinfachen?

Aufgabe 4 (Rucksackproblem/Pipeline)

(12 Punkte)

In dieser Aufgaben parallelisieren Sie das Rucksackproblem mit Hilfe von Geometrischer Dekomposition und Pipelining. Für die effiziente Lösung des Problems verwendet man Dynamische Programmierung und speichert Zwischenergebnisse in einer Tabelle. Zum Befüllen der Tabelle wird für jede Zelle ein Maximum aus der linken Nachbarzelle, der Zelle darüber und einem „Einpackwert“ berechnet (soweit vorhanden). Der „Einpackwert“ berechnet sich als Summe des Gegenstandswertes der aktuellen Spalte und einer diagonalen Zelle (eine Spalte nach links und entsprechend dem Gewicht des Gegenstands viele Zellen nach oben, siehe Abbildung).

	Obj1	Obj2	Obj3	Obj4	Obj5	Obj6
Wert	4	1	2	5	8	2
Gewicht	3	2	1	3	5	1
Max. Gew.	DP-Tabelle					
0	0	0	0	0	0	0
1	0	0	2	2	2	2
2	0	1	2	2	2	4
3	4	4	4	5	5	5
4	4	4	6	7	7	7
5	4	5	6	7	8	9
6	4	5	7	9	10	10
7	4	5	7	11	11	12
8	4	5	7	11	13	13
		Spaltenblock 1		Spaltenblock 2	Spaltenblock 3	

- a) Die erzeugten Aktivitäten füllen die Zellen ihres Spaltenblocks zeilenweise.

Was muss die Aktivität I bereits ausgerechnet haben, damit die Aktivität $I+1$, die für den nachfolgenden Spaltenblock zuständig ist (= die nächste Pipeline-Stufe), frühestens anfangen kann, ihre 3. Zeile zu befüllen?

- b) In der folgenden Implementierung soll jede Pipeline-Stufe mit einer **Queue** abschließen, in die die Aktivität pro Zeile ein Objekt (**READY**) als Fertigmeldung einstellt. Die nachfolgende Aktivität kann arbeiten, sobald die entsprechende Fertigmeldung vorliegt. Vervollständigen Sie die parallele Implementierung des Rucksackproblems (ohne Ausnahmebehandlung). Die nicht ausprogrammierte Methode `calcValue` berechnet nach dem Schema aus der obigen Abbildung den Wert für eine Zelle der Tabelle.
- Sorgen Sie in dem mit 1 markierten Feld dafür, dass die einzelnen Pipeline-Stufen möglichst gleich viele Spalten der Tabelle füllen. Es müssen alle Spalten gefüllt werden.
 - Realisieren Sie in den mit 2 markierten Feldern die notwendige Synchronisation über die Queues.
 - Garantieren Sie in dem mit 3 markierten Feld, dass der von `knapsack` zurückgegebene Wert korrekt berechnet wurde.

```
import java.util.concurrent.*;

public class Knapsack {

    final static Object READY = new Object();

    final BlockingQueue<Object>[] queues;

    final int maxWeight, itemCount, threads;

    final int[][] table;

    final int[] weights, values;

    public Knapsack(int[] values, int[] weights, int maxW, int threads) {
        this.values = values;
        this.weights = weights;
        this.maxWeight = maxW;
        this.threads = threads;

        itemCount = weights.length;
        queues = new BlockingQueue[threads];
        table = new int[maxWeight + 1][itemCount];
    }

    private int calcValue(int weight, int item) {...}
```

```

public int knapsack() throws Exception {
    for (int i = 0; i < threads; i++) {
        final int threadId = i;
        queues[threadId] = new LinkedBlockingQueue<Object>();
    }

    final int start =
    final int end =

    Thread t = new Thread() { public void run() { try {
        for (int w = 1; w <= maxWeight; w++) { // zeilenweise

        }

        for (int i = start; i < end; i++) { // fuelle Zeile des Blocks
            table[w][i] = calcValue(w, i);
        }

        }
    } catch (InterruptedException e) {} }};
    t.start();
}

return table[maxWeight][k.itemCount - 1];
} // Ende knapsack
} // Ende class Knapsack

```

Aufgabe 6 (Levenshtein-Distanz)

(12 Punkte)

In dieser Aufgaben parallelisieren Sie den Algorithmus zur Berechnung der Levenshtein-Distanz mit Hilfe von Geometrischer Dekomposition und Pipelining. Für die effiziente Lösung des Problems verwendet man Dynamische Programmierung und speichert Zwischenergebnisse in einer Tabelle. In der parallelen Version füllen die Aktivitäten die Zellen ihres Spaltenblocks zeilenweise. Jede Zeile entspricht dabei einer Pipeline-Stufe.

	ϵ	H	A	B	I	T
ϵ	0	1	2	3	4	5
H	1	0	1	2	3	4
O	2	1	1	2	3	4
B	3	2	2	1	2	3
B	4	3	3	2	2	3
I	5	4	4	3	2	3
T	6	5	5	4	3	2

Spaltenblock 1
Spaltenblock 2
Spaltenblock 3

ϵ entspricht dem leeren Wort

In der folgenden Implementierung der Methode `levenshtein()` soll jede Pipeline-Stufe mit einer **Queue** abschließen, in die die Aktivität pro Zeile ein Objekt (**READY**) als Fertigmeldung einstellt. Die nachfolgende Aktivität kann arbeiten, sobald die entsprechende Fertigmeldung vorliegt.

Vervollständigen Sie die parallele Implementierung des Levenshtein-Algorithmus (ohne Ausnahmebehandlung). Die nicht ausprogrammierte Methode `calcValue` berechnet für die Zeichenketten x und y nach folgender Definition den Wert für eine Zelle der Tabelle **T**: Für $T_{i,j}$ und $1 \leq i < rows, 1 \leq j < columns$ gilt:

$$\begin{aligned} \text{Im Fall } y_i = x_j: \quad & T_{i,j} = T_{i-1,j-1} + 0 \\ \text{Im Fall } y_i \neq x_j: \quad & T_{i,j} = \min \begin{cases} T_{i-1,j-1} + 1 \\ T_{i,j-1} + 1 \\ T_{i-1,j} + 1 \end{cases} \end{aligned}$$

Nach der Berechnung der Matrix lässt sich die Levenshtein-Distanz an der Stelle $T_{rows-1,columns-1}$ ablesen.

- Sorgen Sie in dem mit 1 markierten Feld dafür, dass die einzelnen Pipeline-Stufen möglichst gleich viele Spalten der Tabelle füllen. Es müssen alle Spalten gefüllt werden. Die Anzahl der Threads kann die Anzahl der Spalten übersteigen.
Hinweis: `calcValue` soll nicht für die Zellen in der ersten Spalte (`c = 0`) bzw. ersten Zeile (`row = 0`) aufgerufen werden.
- Realisieren Sie in den mit 2 markierten Feldern die notwendige Synchronisation über die Queues.
- Garantieren Sie in dem mit 3 markierten Feld, dass der von `levenshtein()` zurückgegebene Wert korrekt berechnet wurde.

```
import java.util.concurrent.*;
public class Levenshtein {

    final static Object READY = new Object();
    final BlockingQueue<Object>[] queues;
    final int threads, rows, columns;
    final int[][] table;
    final char[] wordX, wordY;

    public Levenshtein(char[] wordX, char[] wordY, int threads) {
        this.wordX = wordX;
        this.wordY = wordY;
        this.rows = wordY.length + 1;
        this.columns = wordX.length + 1;
        this.threads = threads;
        queues = new BlockingQueue[threads];
        table = new int[rows][columns];

        for (int r = 0; r < rows; r++){
            table[r][0] = r;
        }
        for (int c = 0; c < columns; c++){
            table[0][c] = c;
        }
    }
    private int calcValue(int row, int column) {...}
```

```

public int levenshtein() throws Exception {
    for (int i = 0; i < threads; i++) { // for #1
        final int threadId = i;
        queues[threadId] = new LinkedBlockingQueue<Object>();

```

1

```

        final int start =
        final int end =

```

```

        Thread t = new Thread() { public void run() { try {
            for (int row = 1; row < rows; row++) {

```

2

```

                for (int c = start; c < end; c++) { // fuelle Zeile des Blocks
                    table[row][c] = calcValue(row, c);
                }

```

2

```

            }
        } catch (InterruptedException e) {}
        t.start();
    } // Ende for #1

```

3

```

    return table[rows-1][columns-1];
} // Ende levenshtein
} // Ende class Levenshtein

```

Aufgabe 3 (ExecutorService)

(12 Punkte)

In dieser Aufgabe soll eine **ExecutorService**-Implementierung vervollständigt werden. Diese Implementierung wird zum Test in einem Beispielprogramm zur Berechnung der Summe aller Elemente eines Arrays eingesetzt.

- a) Sorgen Sie in dem mit 1 markierten Feld dafür, dass die korrekte Anzahl (=packets) an **SumC**-Objekten erstellt wird. Übergeben Sie diese Objekte an den Executor. Passen Sie die Grenzen [begin,end) so an, dass möglichst alle Arbeitspakete die gleiche Größe haben.
- b) In dem mit 2 markierten Feld soll die **run**-Methode der im Executor verwendeten Threads ergänzt werden. Untätige Threads sollen unbearbeitete Arbeitsaufträge aus der **queue** nehmen (**queue.poll()**) und diese ausführen. Beachten Sie, dass **queue.poll()** nicht blockiert, wenn kein Arbeitspaket vorhanden ist. Die Threads dürfen sich erst beenden, wenn **shutdown == true** und **queue.empty()** gilt.
- c) Anschließend muss im Feld 3 die **submit**-Methode implementiert werden. Solange der Executor nicht heruntergefahren wurde, sollen neue Arbeitspakete akzeptiert und per **queue.add** zur Bearbeitung aufgenommen werden.

```
import java.util.concurrent.*;
public class ArraySum {
    public static class SumC implements Callable<Long> {
        public SumC(long[] array, int begin, int end) { ... }
    }
    public static long sum(long[] array, int packets) throws Exception {
        MyExecutor e = new MyExecutor(packets);
        long tmp = 0;
        Future<Long>[] future = new Future[packets];
```

1

```
        e.shutdown();
        for (int i = 0; i < packets; i++) {
            tmp += future[i].get();
        }
        return tmp;
    }
}
```

```
public class MyExecutor {  
    public LinkedBlockingQueue<Runnable> queue =  
        new LinkedBlockingQueue<Runnable>();  
    public MyThread[] threads;  
    public volatile boolean shutdown = false;  
  
    public class MyThread extends Thread {  
        public void run() {
```

2

```
        }  
    }  
}
```

```
public MyExecutor(int nThreads) {  
    threads = new MyThread[nThreads];  
    for (int i = 0; i < nThreads; i++) {  
        threads[i] = new MyThread();  
        threads[i].start();  
    }  
}
```

```
public <T> Future<T> submit(Callable<T> task) {  
    FutureTask<T> f = new FutureTask<T>(task);
```

3

```
        return null;  
    }
```

```
    public void shutdown() {  
        shutdown = true;  
    }  
}
```

Aufgabe 5 (Zeigerverdopplung)

(11 Punkte)

Vervollständigen Sie die Klasse **PointerDoubling**, die mit dem parallelen Algorithmus der Zeigerverdopplung den Rang aller Elemente einer linearen Liste, d.h. ihren Abstand vom Ende der Liste, berechnet. Die Verkettung der Listenelemente wird mithilfe des **int**-Arrays **next** abgebildet. Der Eintrag **next[i]** enthält den Index des Nachfolgers des Elements **i**. Der Eintrag für das letzte Element ist **-1**.

Hinweis: Bei dem Verfahren wird jedes Element von einem Thread verarbeitet.

Zur Erinnerung der Verfahrensablauf:

- Initialisiere den Rang des letzten Listenelements mit 1.
- Setze Distanz **distance = 1**.
- Solange noch nicht jedes Element einen Rang hat:
 - Wenn die **next**-Referenz des Elements **i** auf ein Element mit Rang **rank[next[i]] != 0** zeigt, dann setze den Rang auf den Wert **rank[next[i]] + distance** und die Referenz auf **0**.
 - Sonst setze die **next**-Referenz auf die des bisherigen Nachfolgers.
 - Verdopple die Distanz.

- a) Teilen Sie in dem mit 1 markierten Feld die Elemente des Arrays `next` auf **Worker**-Threads auf und aktivieren Sie diese Aktivitätsfäden. Stellen Sie eine korrekte parallele Verarbeitung durch die erzeugten Threads sicher.
- b) Führen Sie im mit 2 markierten Feld die Initialisierung des Ranges des letzten Elements der Liste durch.
- c) Nutzen Sie das Feld 3 zur Berechnung der Werte `next[index]` und `rank[index]`.
- d) Nutzen Sie in den mit 4 bezeichneten Feldern die **CyclicBarrier** `inner` zur Synchronisation des Ablaufes der parallelen Verarbeitung der **Worker**-Threads.
Hinweis: Die Barriere `inner` kann bei Bedarf in mehreren mit 4 bezeichneten Feldern genutzt werden.

```
import java.util.concurrent.*;
```

```
public class PointerDoubling {
    private static int[] next;
    private static int[] rank;

    private static CyclicBarrier outer;
    private static CyclicBarrier inner;

    public static int[] calculateRanks(int[] next)
        throws InterruptedException, BrokenBarrierException {
        PointerDoubling.next = next;
        rank = new int[next.length];
        outer = new CyclicBarrier(next.length + 1);
        inner = new CyclicBarrier(next.length);
```

1

```
        outer.await();
        return rank;
    }
```

```
    public static class Worker extends Thread {
        private final int index;
        public Worker(int index) {
            this.index = index;
        }
    }
```

```
public void run() {
    try {
```

2

4

```
        int distance = 1;
```

```
        while (distance < next.length) {
            int rankOfNext = 0;
            int nextOfNext = 0;
            if (next[index] > 0) {
                rankOfNext = rank[next[index]];
                nextOfNext = next[next[index]];
            }
        }
```

4

3

4

```
            distance *= 2;
        } //END OF while
        outer.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        e.printStackTrace();
    }
} //END OF run
} //END OF Worker
} //END OF PointerDoubling
```

Aufgabe 3 (Präfixsumme)

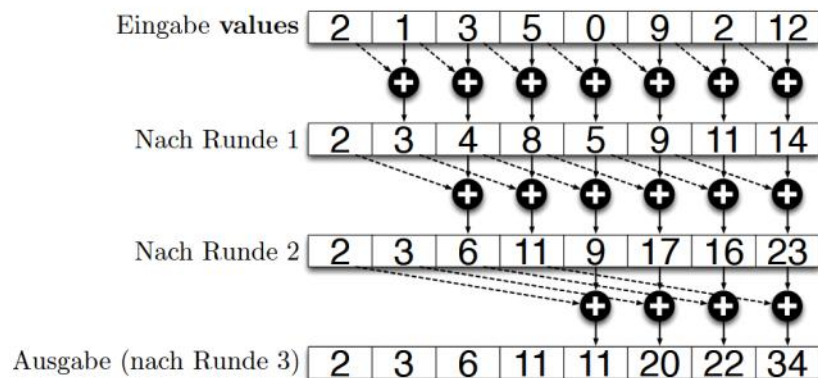
(12 Punkte)

Es ist ein `int`-Array `values` gegeben. Die Präfixsumme für eines der Elemente dieses Arrays ist die Summe aller Elemente vom Beginn des Arrays bis zu dem ausgewählten Element (einschließlich).

Vervollständigen Sie die Klasse `ParallelPrefixSums` zur Berechnung der Präfixsummen **aller** Werte des übergebenen `int`-Arrays `values`. Dabei wird für jedes Element `i` ein separater Thread gestartet, der folgende Schritte ausführt:

- I. Setze Distanz `distance = 1`.
- II. Solange die Distanz `distance` kleiner als die Länge des Arrays `values` ist:
 1. Warte an der Barriere `barrier` auf die anderen Threads.
 2. **Berechne den Zielindex `target = i + distance` der aktuellen Runde.**
 3. **Falls der Zielindex gültig ist, dann addiere das Element `value[i]` zum Element `value[target]` hinzu.**
 4. Verdopple die Distanz `distance`.

Das Verfahren arbeitet direkt in dem übergebenen Array `values`. Beispiel:



- a) Welchem Ihnen aus der Vorlesung bekannten Verfahren ähnelt der oben beschriebene Algorithmus?

- b) Initialisieren Sie in dem mit 1 markierten Feld die Barriere `barrier`.
- c) Teilen Sie in dem mit 2 markierten Feld die Elemente des Arrays `values` auf `Worker`-Threads auf und aktivieren Sie die erzeugten Aktivitätsfäden.
- d) Stellen Sie in dem mit 3 markierten Feld das korrekte Beenden des Programms mit dem gewünschten Ergebnis sicher.
- e) Implementieren Sie die **oben fettgedruckten Schritte II.2 und II.3** des Algorithmus in dem mit 4 markierten Feld.

Hinweis: Vermeiden Sie Wettlaufsituationen zwischen parallelen Lese- und Schreibzugriffen.

```

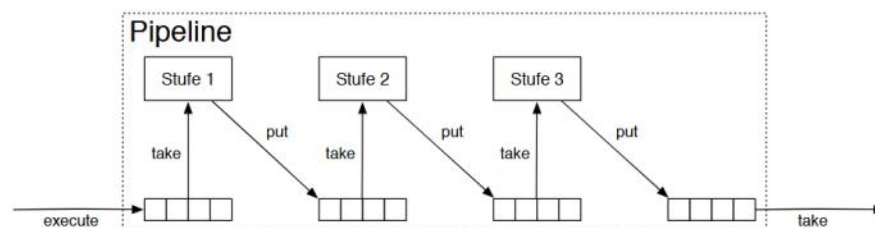
import java.util.concurrent.*;
public class ParallelPrefixSums {
    private static CyclicBarrier barrier;
    public static void calculateSums(int[] values) throws InterruptedException {
1
        barrier =
    Thread[] workers = new Thread[values.length];
2
    for (
    ) {
    }
3
    } //END OF FOR
    for (int i = 0; i < workers.length; i++) {
    }
4
    } //END OF FOR
    public static class Worker implements Runnable {
        private int[] values;
        private int index;
        public Worker(int[] values, int index) { ... //usual assignments }
        public void run() {
            try {
                int distance = 1;
                while (distance < values.length) {
                    barrier.await(); //Schritt II.1
                }
                if (
                ) {
                }
            } //END OF IF
            distance *= 2; //Schritt II.4
        } //END OF WHILE
    } catch (InterruptedException | BrokenBarrierException e) {
        e.printStackTrace();
    } //END OF TRY
    } //END OF RUN
    } //END OF WORKER
}

```

Aufgabe 3 (Pipeline)

(13 Punkte)

In dieser Aufgabe vervollständigen Sie die Klasse `Pipeline`, die das Pipeline-Muster implementiert. Eine Pipeline hat `nStages` viele Stufen, die jeweils von einem `StageThread` bearbeitet werden. Die Threads kommunizieren über die `nStages + 1` Schlangen des Arrays `queues` (siehe Abbildung für eine Pipeline mit 3 Stufen). Mit der Methode `execute` werden Aufträge an die erste Schlange der Pipeline übergeben. Die Ergebnisse der Berechnungen der Pipeline werden in der letzten Schlange abgelegt und können dort vom Aufrufer abgeholt werden. Eine Pipeline wird durch den Aufruf der Methode `setup` initialisiert. Durch den Aufruf der Methode `shutdown` wird die Pipeline heruntergefahren, indem die Konstante `POISON_PILL` in die erste Schlange gelegt wird. Jeder `StageThread` beendet seine Verarbeitung, sobald er diese Konstante als Eingabe erhält.



- a) Initialisieren und starten Sie in den mit 1 markierten Feldern die Threads vom Typ `StageThread` und verbinden Sie diese über die zugehörigen Schlangen. Speichern Sie die Referenzen der erstellten Threads im Array `threads`.
- b) Vervollständigen Sie nun die Implementierung der `run`-Methode in den mit 2 bis 4 markierten Feldern:
 - i) Dazu sollen in dem mit 2 markierten Feld die Werte aus der eingehenden Schlange gelesen werden.
 - ii) Führen Sie in dem mit 3 markierten Feld die normale Verarbeitung einer Pipelinestufe aus, indem Sie die Methode `runStage` aufrufen und das Ergebnis an die nächste Pipelinestufe weitergeben.
 - iii) Implementieren Sie in dem mit 4 markierten Feld das korrekte Beenden eines `StageThreads`. Achten Sie darauf, dass die nachfolgenden Stufen ebenfalls beendet werden müssen. Vermeiden Sie, dass die Konstante `POISON_PILL` an die letzte Schlange übergeben wird (da sie sonst als Ausgabe der Pipeline gewertet würde).

```

import java.util.concurrent.*;
public class Pipeline {
    public static final Integer POISON_PILL = Integer.MIN_VALUE;

    private BlockingQueue<Integer>[] queues;
    private Thread[] threads;

    public BlockingQueue<Integer> setup(int nStages) {
        queues = new BlockingQueue[nStages + 1];
        for (int i = 0; i < nStages + 1; i++) {
            queues[i] = new LinkedBlockingQueue<Integer>();
        }
        threads = new Thread[nStages];

        for (int i = 0; i < threads.length; i++) {
            threads[i] = new Thread(new Worker(i, queues));
            threads[i].start();
        }

        boolean lastThread = (i == threads.length - 1);

        BlockingQueue<Integer> inQueue = queues[i];

        BlockingQueue<Integer> outQueue = queues[i + 1];

        return outQueue;
    }

    return queues[queues.length - 1];
} //END OF SETUP

public void execute(Integer value) { /* push value to first queue */ }

public void shutdown() { /* push POISON_PILL to first queue */ }
} //END OF CLASS

```

```

import java.util.concurrent.*;

public class StageThread extends Thread {

    private int id;
    private BlockingQueue<Integer> inQueue;
    private BlockingQueue<Integer> outQueue;
    private boolean lastThread;

    StageThread(int id, BlockingQueue<Integer> inQueue,
        BlockingQueue<Integer> outQueue, boolean lastThread) {
        /* assignments */
    }

    public void run() {
        while (true) {
            try {
                Integer value =
            }
            if (!value.equals(Pipeline.POISON_PILL)) {
            }
            } else {
            }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    } //END OF RUN
    // runs the correct stage for the id of this thread
    private Integer runStage(Integer value) {
        switch (this.id) { /* ... */ }
    }
} //END OF CLASS

```

2

3

4

Aufgabe 3 (Datei-Suche)

(15 Punkte)

Es soll ein Programm entwickelt werden, das alle Dateien eines Verzeichnisbaumes (also auch Dateien in Unterverzeichnissen beliebiger Tiefe, jedoch keine symbolischen Links) nach einer gegebenen Zeichenfolge parallel durchsucht:

- Wird ein Unterverzeichnis gefunden, soll dieses von einem eigenen Aktivitätsfaden (Instanz von `FolderSearcher`) durchsucht werden. Die Anzahl solcher Threads ist unbegrenzt.
- Pro gefundener Datei soll ein eigener Aktivitätsfaden (`FileSearcher`) zur zeilenweisen Bearbeitung verwendet werden. Ein Threadpool stellt dazu maximal `maxFileThreads` Fäden bereit.

Zur Darstellung von Datei- und Verzeichnispfaden wird die Java-Klasse `File` verwendet. Beide Arten an Threads erhalten in ihrem Konstruktor eine Referenz auf ein `File`-Objekt. Die Implementierung der `FileSearcher` und die ErgebnISRückgabe sind nicht Teil dieser Aufgabe.

- a) Die Methode `grep(String search, File rootDir, int maxFileThreads)` stößt die parallele Suche nach dem String `search` mit maximal `maxFileThreads` gleichzeitig aktiven `FileSearcher`-Threads an. Als Ausgangspunkt der Suche dient das Verzeichnis `rootDir`. Initialisieren Sie das Attribut `activeFolderSearchers` der Klasse zum Zählen der aktiven `FolderSearcher` und starten Sie den ersten `FolderSearcher` im mit **1** markierten Feld.

```
import java.io.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
```

```
public class ParallelGrep {
```

```
    private ExecutorService pool;
    private AtomicInteger activeFolderSearchers; // for atomic counting
    private String search; // FileSearchers need to know what to search
```

```
    public void grep(String search, File rootDir, int maxFileThreads)
        throws InterruptedException {
        this.search = search;
        pool = Executors.newFixedThreadPool(maxFileThreads);
```

```
        activeFolderSearchers =
```

1

```
        pool.awaitTermination(1, TimeUnit.HOURS);
```

```
    }
```

```
    class FileSearcher implements Runnable { // inner class of ParallelGrep
        private File file;
        public FileSearcher(File file) {
            this.file = file;
        }
        // ...
    } // ENDE FileSearcher
```

- b) Implementieren Sie nun die Methode `run()` in `FolderSearcher` in Feld **2**. Erstellen und aktivieren Sie je nach Art des gefundenen `File` eine neue Instanz des entsprechenden Bearbeiters. Denken Sie auch an das korrekte Erhöhen von `activeFolderSearchers`. Folgende Methoden der Klasse `File` könnten für diese Aufgabe relevant sein:

<code>boolean isDirectory()</code>	Prüft, ob das <code>File</code> -Objekt auf ein Verzeichnis verweist.
<code>boolean isFile()</code>	Prüft ob das <code>File</code> -Objekt auf eine Datei verweist.

- c) Bevor sich der **letzte** `FolderSearcher` beendet, muss er den Threadpool der `FileSearcher` mittels `shutdown()` beenden. Modifizieren und prüfen Sie hierzu `activeFolderSearchers` in Feld **3** der `run()`-Methode.

```
class FolderSearcher extends Thread { // inner class of ParallelGrep
    File root;

    public FolderSearcher(File root) {
        this.root = root;
    }

    @Override
    public void run() {
        File[] files = root.listFiles();
        for (File file : files) {
```

2

```
        } // ENDE for
```

```
        if (
        ) {

            pool.shutdown(); // Shall only be called once!
        }
    }
}
```

3

```
    } // ENDE run()
} // ENDE FolderSearcher
} // ENDE ParallelGrep
```

Aufgabe 2 (Longest Common Subsequence)

(12 Punkte)

In dieser Aufgabe parallelisieren Sie den Algorithmus zur Berechnung der Longest Common Subsequence (LCS) mit Hilfe von Geometrischer Dekomposition und einem Fließband. Für die effiziente Lösung des Problems verwendet man Dynamische Programmierung und speichert Zwischenergebnisse in einer Tabelle. Jeder Thread bearbeitet eine Fließbandstufe und füllt die Werte seines Bereichs von oben nach unten zeilenweise.

In der folgenden Implementierung der Methode `lcs()` soll jede Fließbandstufe mit einer Queue abschließen. Jeder Thread soll nach der Bearbeitung einer Zeile ein Objekt (READY) als Fertigmeldung in die passende Queue einstellen. Der nachfolgende Thread kann arbeiten, sobald die entsprechende Fertigmeldung vorliegt.

	ϵ	H	A	B	I	T
ϵ	0	0	0	0	0	0
H	0	1	1	1	1	1
O	0	1	1	1	1	1
B	0	1	1	2	2	2
B	0	1	1	2	2	2
I	0	1	1	2	3	3
T	0	1	1	2	3	4
Thread 0			Thread 1		Thread 2	

ϵ entspricht dem leeren Wort

Vervollständigen Sie die parallele Implementierung des LCS-Algorithmus (ohne Ausnahmebehandlung). Die Methode `calcValue` berechnet für die Zeichenketten `x` und `y` nach folgender Definition den Wert für eine Zelle der Tabelle `T`:

Für $T_{i,j}$ und $1 \leq i < rows, 1 \leq j < columns$ gilt:

$$\text{Im Fall } y_i = x_j: \quad T_{i,j} = T_{i-1,j-1} + 1$$

$$\text{Im Fall } y_i \neq x_j: \quad T_{i,j} = \max \begin{cases} T_{i,j-1} \\ T_{i-1,j} \end{cases}$$

Nach der Berechnung der Matrix lässt sich die LCS durch den Aufruf von `readMatrix` ablesen.

- a) Sorgen Sie in dem mit 1 markierten Feld auf der nächsten Seite dafür, dass die einzelnen Fließbandstufen möglichst gleich viele Spalten der Tabelle füllen. Es müssen alle Spalten gefüllt werden. Die Anzahl der Spalten übersteigt die Anzahl der Threads. Sorgen Sie bei der Aufteilung dafür, dass `calcValue` nicht für die Zellen der Spalte 0 aufgerufen wird.
- b) Realisieren Sie in den beiden mit 2 markierten Feldern die notwendige Synchronisation zwischen den Fließbandstufen mit Hilfe der Queues.
- c) Stellen Sie in dem mit 3 markierten Feld sicher, dass `lcs()` erst beendet wird und einen Wert zurück liefert, wenn die Fließbänder ihre Arbeit beendet haben.

```
import java.util.concurrent.*;
public class LCS {

    final static Object READY = new Object();
    final BlockingQueue<Object>[] queues;
    final int threads, rows, columns;
    final int[][] table;
    final char[] wordX, wordY;

    public LCS(char[] wordX, char[] wordY, int threads) {
        this.wordX = wordX;
        this.wordY = wordY;
        this.rows = wordY.length + 1;
        this.columns = wordX.length + 1;
        this.threads = threads;
        queues = new BlockingQueue[threads];
        table = new int[rows][columns];

        for (int r = 0; r < rows; r++){
            table[r][0] = 0;
        }
        for (int c = 0; c < columns; c++){
            table[0][c] = 0;
        }
    }
    private char[] readMatrix() {...}
    private int calcValue(int row, int column) {...}
```

```

public char[] lcs() throws Exception {
    for (int i = 0; i < threads; i++) { // Anfang der for-Schleife #1
        final int threadId = i;
        queues[threadId] = new LinkedBlockingQueue<Object>();

```

1

```

        final int start =
        final int end =

```

```

        Thread t = new Thread() { public void run() { try {
            for (int row = 1; row < rows; row++) {

```

2

```

                for (int c = start; c < end; c++) { // fuelle Zeile des Blocks
                    table[row][c] = calcValue(row, c);
                }

```

2

```

            }
        } catch (InterruptedException e) {}
        t.start();
    } // Ende der for-Schleife #1

```

3

```

        return readMatrix();
    } // Ende lcs
} // Ende class LCS

```

Aufgabe 4 (Monte-Carlo-Simulation)

(12 Punkte)

Die Methode `pi` berechnet die Zahl π durch eine parallele Monte-Carlo-Simulation. Sie verwendet dafür `Callable`-Objekte, die von einem `ExecutorService` bearbeitet werden. Vervollständigen Sie die folgende Implementierung der Simulation.

- a) Erzeugen Sie in dem mit 1 markierten Feld die erforderlichen Objekte (Anzahl: `tasks`) vom Typ `MonteCarloTask` und übergeben Sie diese an den `ExecutorService` zur Bearbeitung. Die `id` des `MonteCarloTask` gibt zugleich an, welcher der insgesamt `globalCnt` Punkte von dem Task-Objekt zuerst simuliert wird.
- b) Implementieren Sie in dem mit 2 markierten Feld die Zusammenführung der Teilergebnisse in der Variable `hits`.
- c) Implementieren Sie in Feld 3 die eigentliche Simulation. Verwenden Sie `nextDouble` zur Berechnung von Zufallspositionen. Ein Punkt (x,y) ist innerhalb des Einheitskreises, wenn $x^2 + y^2 \leq 1$ gilt. Überlegen Sie sich eine geschickte Schleifenschrittweite.

```
import java.util.Random;
import java.util.concurrent.*;
public class MonteCarlo {
    public static double pi(int tasks, int globalCnt) throws Exception {
        ExecutorService exec = Executors.newFixedThreadPool(tasks);
        Future<Integer>[] hitArray = new Future[tasks];
```

1

```
int hits = 0;
```

2

```
exec.shutdown();
return (((double) hits) / globalCnt) * 4;
} // Ende pi
```

```
static class MonteCarloTask implements Callable<Integer> {  
    private Random rnd;  
    private int id;  
    private int tasks;  
    private int globalCnt;  
  
    public MonteCarloTask(int id, int tasks, int globalCnt) {  
        this.id = id; // des Task-Objekts und des 1. simulierten Punktes  
        this.tasks = tasks;  
        this.globalCnt = globalCnt;  
        this.rnd = new Random();  
    }  
  
    private double nextDouble() {  
        return rnd.nextDouble();  
    }  
  
    public Integer call() {  
        int mySum = 0;
```

```
        return mySum;  
    } // Ende call  
} // Ende class MonteCarloTask  
} // Ende class MonteCarlo
```

3

Aufgabe 4 (Counting)

(13 Punkte)

Ein byte kann 256 Werte (von -128 bis 127) annehmen. Gegeben sei ein byte-Array. Sie sollen einen parallelen Algorithmus entwickeln, der zählt, wie oft jeder Wert tatsächlich in der Eingabe (array) vorkommt.

Vervollständigen Sie die Klassen Counting und CountingThread. Es sollen insgesamt threads viele CountingThreads gestartet werden. Diese ermitteln lokale Ergebnisse, die anschließend in einem globalen AtomicIntegerArray zusammengeführt werden. Synchronisieren Sie den gesamten Ablauf mit Hilfe einer CyclicBarrier.

Auszug möglicherweise relevanter Methoden aus der API der Klasse AtomicIntegerArray:

```
addAndGet(int i, int delta): Atomically adds the given delta to the element at
                             index i and returns the updated value.
get(int i):                  Gets the current value at position i.
set(int i, int newValue):    Sets the element at position i to newValue.
```

- 1) Übergeben Sie in dem mit 1 markiertem Feld die korrekte Anzahl beteiligter Threads an die CyclicBarrier. Bedenken Sie, dass ggf. nicht nur die CountingThreads synchronisiert werden müssen.
- 2) Initialisieren und starten Sie in dem mit 2 markiertem Feld die CountingThreads. Achten Sie auf eine gleichmäßige und vollständige blockweise Aufteilung des Arrays array auf die Threads. Es gilt $\text{threads} \leq \text{array.length}$.
- 3) Überführen Sie in dem mit 3 markierten Feld das Zwischenergebnis eines einzelnen Threads in das globale AtomicIntegerArray globalCounts.
- 4) Synchronisieren Sie, **sofern nötig**, in den beiden mit 4 markierten Feldern den Gesamtablauf mit Hilfe der Barriere.

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicIntegerArray;

public class Counting {
    final byte[] array;
    int threads;

    final AtomicIntegerArray globalCounts;
    final CyclicBarrier barrier;

    Counting(byte[] array, int threads) {
        this.array = array;
        this.threads = threads;
        this.globalCounts = new AtomicIntegerArray(256);

        this.barrier = new CyclicBarrier(
    }
}
```

1

```
AtomicIntegerArray count() throws Exception {
    int step = array.length / threads;
    int start = 0;
```

```
    for (int i = 0;                                ; i++)
```

2

```
    return globalCounts;
}
```

```
class CountingThread extends Thread {
    private int begin, end;
    CountingThread(int begin, int end) {
        this.begin = begin;
        this.end = end;
    }
```

```
    public void run() {
        try {
            // local counting phase -- already implemented
            int[] count = new int[256];
            for (int i = begin; i < end; i++) {
                count[array[i] + 128]++;
            }
            // add values of count to globalCounts
```

3

```
        } catch (Exception e) { /* handle exception */ }
```

```
    }
```

```
}
```

```
}
```

4

Aufgabe 2 (Gauß)

(13 Punkte)

In dieser Aufgabe parallelisieren Sie einen zweistufigen Gauß-Filter mit Hilfe von Geometrischer Dekomposition. Die Spalten des Bildes (`src`) werden so wie im unten gezeigten Beispiel auf `n` Threads aufgeteilt und unabhängig bearbeitet. Für jeden Bildpunkt (`src[row][col]`) wird durch die Methode `step()` ein neuer Wert berechnet und das Ergebnis in `dest` gespeichert. Danach wird von einem Thread die Methode `swap()` aufgerufen, die das Quellbild `src` mit dem Zielbild `dest` vertauscht. Anschließend rufen alle Threads die Methode `foo()` auf. In den folgenden Teilaufgaben vervollständigen Sie die parallele Implementierung des Algorithmus.

Hinweis: Das zu verarbeitende Bild ist quadratisch.

- Bereiten Sie die erforderliche Synchronisationsbarriere in Feld 1 vor.
- Im mit 2 markierten Feld soll die parallele Verarbeitung gestartet werden. Erstellen Sie dafür `nThreads` Objekte vom Typ `GRunnable` und bringen Sie diese mit Hilfe von genau `nThreads` Threads zur parallelen Ausführung.
- Führen Sie in dem mit 4 markierten Feld die Methode `step()` auf jedem Bildpunkt aus. Achten Sie darauf, dass jeder Thread eine Spalte komplett bearbeitet, bevor er mit der nächsten Spalte beginnt.
- Sorgen Sie im mit 5 markierten Feld dafür, dass, nachdem alle Threads ihre Bearbeitung von `step()` abgeschlossen haben, genau einer der Threads die Methode `swap()` ausführt. Erst danach dürfen alle Threads die Bearbeitung von `foo()` beginnen.
- Vervollständigen Sie die Synchronisation in den mit 3 und 6 markierten Feldern. Stellen Sie dabei auch sicher, dass `apply()` erst beendet wird, wenn alle Threads ihre Arbeit beendet haben.

Quellbild `src`:

0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1
0	1	2	3	0	1	2	3	0	1

Beispiel: Aufteilung von `step()` mit 4 Threads (durchnummeriert von 0 bis 3)

```
import java.util.concurrent.*;
public class Gauss {
    private class MyColor {
        float red, green, blue;
    }
    public MyColor[][] src, dest;
    public int size;

    private MyColor step(int row, int col) {...}
    private void foo() {...}
    private void swap() {...}

    public MyColor[][] apply(MyColor[][] image, int nThreads) {
        try {
            src = image;
            size = image.length;
            dest = new MyColor[size][size];
```

```
CyclicBarrier barrier =
```

```
    } catch (Exception e) { }
    return dest;
} // END OF apply
```

```
// INNER CLASS GRunnable:
private class GRunnable implements Runnable {
    int id;
    int nThreads;
    CyclicBarrier barrier;

    public GRunnable(int id, int nThreads, CyclicBarrier barrier) {
        this.id = id;
        this.nThreads = nThreads;
        this.barrier = barrier;
    }

    public void run() {
        try {
```

4

```
            for (int col = 0; col < size; col++) {

                for (int row = 0; row < size; row++) {

                    dest[ row ][ col ] =

                }
            }
        }
    }
```

5

```
        foo();
```

6

```
    } catch (Exception e) { }
} // End run
} // End class GRunnable
} // End class Gauss
```

Aufgabe 2 (Web-Crawler)

(15 Punkte)

In dieser Aufgabe entwickeln Sie einen einfachen parallelen Web-Crawler, der Webseiten sowie davon verlinkte Seiten besucht. Dem Web-Crawler werden eine Start-URL sowie die Anzahl der maximal gleichzeitig aktiven `CrawlThreads` übergeben.

Das Ausführungsmodell sei wie folgt: Der Web-Crawler startet zunächst mit einem Thread und aktiviert nach und nach weitere `CrawlThreads`, bis eine Obergrenze an aktiven Threads erreicht ist.

Der Web-Crawler ist konzeptuell in zwei Teile unterteilt. Die Verwaltung der Threads wird von der Methode `manageThreads` übernommen, während die Verarbeitung der URLs in der Methode `process` geschieht. `manageThreads` wird dabei sowohl vom Hauptthread als auch von den `CrawlThreads` aufgerufen.

- a) Die Anzahl der Threads soll durch Zählen der aktiven `CrawlThreads` begrenzt werden. Deklarieren und initialisieren Sie im Feld 1 die erforderlichen Datenstrukturen.
- b) Die Methode `manageThreads` im Feld 2 soll entscheiden, ob für die übergebene `linkUrl` ein neuer Thread erzeugt werden darf, ohne die maximal erlaubte Anzahl (`maxThreads`) zu überschreiten. Ist dies der Fall, soll `true` zurückgegeben werden, andernfalls `false`. Formulieren Sie eine geeignete Bedingung für die `if`-Anweisung. Wenn diese zu `true` ausgewertet wird, erzeugen Sie einen neuen `CrawlThread` und bringen Sie diesen zur parallelen Ausführung.
- c) Passen Sie in der Methode `run` im Feld 3 Ihren Zähler der aktiven Threads entsprechend an.
- d) Der Web-Crawler soll jede URL nur einmal verarbeiten. Dies soll mithilfe der Menge `visitedUrls` erreicht werden. Enthält `visitedUrls` die aktuelle `linkUrl` noch nicht (Methode `contains`), soll diese hinzugefügt werden (Methode `add`) und `doVisit` auf `true` gesetzt werden. Ergänzen Sie entsprechenden, thread-sicheren Code im Feld 4.

Hinweis: Sie können davon ausgehen, dass die Methode `crawl` nicht parallel aufgerufen wird.

```

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

class WebCrawler {
    BlockingQueue<CrawlThread> usedThreads;

    // Speichert besuchte URLs
    Set<String> visitedUrls = new HashSet<String>();

    // Gibt Liste von Links im Inhalt von url zurueck
    List<String> getUrls(String url) { /* ... */ }

    // Verarbeitet url
    void visit(String url) { /* ... */ }

    int maxThreads;

```

1

```

public void crawl(String startUrl, int maxThreads) throws Exception {
    usedThreads = new LinkedBlockingQueue<CrawlThread>();
    visitedUrls.add(startUrl);
    this.maxThreads = maxThreads;

    manageThreads(startUrl);

    for (final CrawlThread ct : usedThreads) {
        ct.join();
    }
} // END of crawl

```

2

```

boolean manageThreads(String linkUrl) {

    if (

        CrawlThread t =

        usedThreads.add(t);
        return true;
    } else {

        return false;
    }
} // END OF manageThreads

```

```

class CrawlThread extends Thread { // innere Klasse
    private String startUrl;

    public CrawlThread(String startUrl) {
        this.startUrl = startUrl;
    }

```

3

```

    public void run() {

        process();

    }

```

```

void process() {
    final Queue<String> urlQueue = new ArrayDeque<>();
    urlQueue.add(startUrl);

    while (!urlQueue.isEmpty()) {
        final String url = urlQueue.remove();
        visit(url); // Verarbeitet url

        for (String linkUrl : getUrIs(url)) {
            boolean doVisit = false;

```

4

```

                if (doVisit) {
                    if (!manageThreads(linkUrl)) {
                        urlQueue.add(linkUrl);
                    }
                }
            } // END of while loop
        } // END of process
    } // END of CrawlThread
} // END of WebCrawler

```

Aufgabe 2 (Paralleler Rätsellöser)

(14 Punkte)

Implementieren Sie im Folgenden einen parallelen Rätsellöser. Ausgehend von einem Zustand (Instanz der Klasse `State`) sucht das Programm nach einer Lösung, indem es überprüft, ob der aktuelle Zustand einen Lösungszustand darstellt. Falls ja, wird dieser Zustand als Lösung zurückgegeben. Andernfalls werden die Nachfolgezustände untersucht.

Hinweis: Sie können davon ausgehen, dass das Rätsel immer mindestens eine Lösung hat und die Methode `solve()` nicht parallel ausgeführt wird. Falls das Rätsel mehrere Lösungen hat, genügt es, eine davon zurückzugeben.

Die parallele Implementierung benutzt eine Warteschlange (`work`) zur Verwaltung der zu untersuchenden Zustände. Zusätzlich darf sie nur Zustände in die Warteschlange legen, die **nicht** nicht vorher erreicht wurden.

- a) Deklarieren und initialisieren Sie im mit 1 markierten Feld je ein Attribut für eine gefundene Lösung (vom Typ `State`) sowie eine Sperre bzw. Barriere, die das Finden einer Lösung signalisiert.
- b) Vervollständigen Sie im mit 2 markierten Feld die Implementierung der Aktivitätsfäden. Solange noch keine Lösung gefunden wurde, nehmen diese sich einen Zustand aus der Warteschlange und bearbeiten diesen. Stellt dieser einen Lösungszustand dar, beenden sich alle Aktivitätsfäden. Andernfalls wird jeder noch nicht besuchte Nachfolgezustand als besucht markiert und in die Warteschlange gelegt. Stellen Sie sicher, dass ein Nachfolgezustand nicht mehrfach in die Warteschlange eingefügt wird.
- c) Bringen Sie im mit 3 markierten Feld `numThreads Worker` zur parallelen Ausführung. Warten Sie dann mithilfe Ihrer Sperre bzw. Barriere, dass diese eine Lösung finden, und geben Sie diese Lösung zurück.

```

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class PuzzleSolver {
    // Warteschlange mit zu bearbeitenden Zuständen:
    BlockingQueue<State> work = new LinkedBlockingQueue<State>();

    // Thread-sichere Menge bereits besuchter Zustände:
    Set<State> visited = Collections.synchronizedSet(new HashSet<>());

```

1

```

// Prüft, ob der Zustand eine Lösung darstellt
boolean isSolution(State st) { /* ... */ }

// Gibt Nachfolgezustände fuer einen Zustand zurueck
List<State> getNextStates(State st) { /* ... */ }

```

```

class Worker implements Runnable {
    public void run() {
        try {

```

2

```

            while (
                State cur = work.poll();

                if (cur != null) {
                    if (isSolution(cur)) {
                        // Ergebnis speichern und Sperre/Barriere betätigen

                    } else {
                        for (State next : getNextStates(cur)) {
                            // Lege jeden noch nicht besuchten Nachfolgezustand in die
                            // Warteschlange
                        }
                    }
                }
            } // END OF while

        } catch (Exception e) {
            /* Ignore */
        }
    } // END OF run()
} // END OF Worker

```

```
} // END OF PuzzleSolver
```

3

Aufgabe 4 (Paralleles Sortieren durch Verschmelzen)

(13 Punkte)

Vervollständigen Sie nachfolgende Umsetzung eines parallelen Sortierens durch Verschmelzen (MergeSort). Dieses teilt eine Reihung in zwei Hälften, auf die das Verfahren rekursiv angewandt wird. Die somit sortierten Hälften werden danach zu einer sortierten Reihung verschmolzen.

Maximal `maxThreads` Worker sollen gleichzeitig arbeiten. Sofern diese Grenze noch nicht erreicht ist, sollen die beiden Hälften vom aktuell betrachteten Teil der Reihung parallel sortiert werden. Andernfalls geschieht die Verarbeitung sequentiell.

- a) Deklarieren Sie in Feld 1 einen Zähler für die arbeitenden Aktivitätsfäden. Initialisieren Sie Ihren Zähler im Feld 2.
- b) Die Methode `mergeSort` enthält das eigentliche Sortierverfahren. Arbeiten weniger als `maxThreads` Aktivitätsfäden, soll ein neuer Worker erzeugt werden, der die erste Hälfte bearbeitet, während der ursprüngliche Aktivitätsfaden parallel die zweite Hälfte übernimmt. Danach sollen die Ergebnisse zusammengeführt werden. Ergänzen Sie Feld 3 entsprechend.
- c) Die Klasse `Worker` implementiert die Aktivitätsfäden. Modifizieren Sie in Feld 4, falls nötig, Ihren Zähler.

Hinweis: Sie können davon ausgehen, dass die Methode `sort` nie parallel ausgeführt wird.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
public class ParallelMergeSort {
```

```
    int[] array;
    int maxThreads;
```

1

```
    public void sort(int[] array, int maxThreads) {
        this.array = array;
        this.maxThreads = maxThreads;
```

2

```
        mergeSort(0, array.length);
    }
```

```
    // Zusammenfuehrung der Teilstuecke, ist gegeben
    void merge(int from, int mid, int to) { ... }
```

```

void mergeSort(int from, int to) {
    if (to - from <= 1) return;
    try {
        int mid = from + (to - from) / 2;
        Worker child = null;

```

3

```

        if (
            ) {

            // Weniger als maxThreads Threads aktiv -> erstelle neuen Worker

            child =

        } else {
            // Es laufen bereits maxThreads Threads -> sequentiell ausfuehren

            mergeSort(from, mid);

        }

        mergeSort(mid, to);

```

```

        merge(from, mid, to);
    } catch (Exception e) {}
} // END OF mergeSort()

class Worker extends Thread {
    int from;
    int to;

    Worker(int from, int to) {
        this.from = from;
        this.to = to;
    }
    public void run() {

```

4

```

        mergeSort(from, to);

```

```

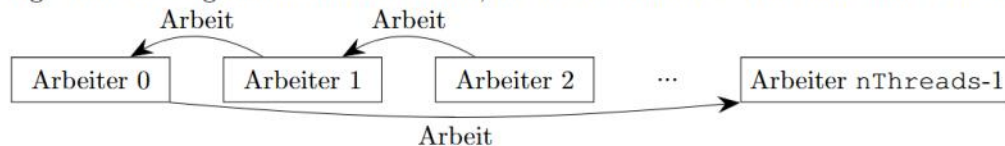
    }
} // END OF Worker
} // END OF ParallelMergeSort

```

Aufgabe 4 (Arbeitsdiebstahl)

(15 Punkte)

Die Methode `map` der Klasse `Mapper` soll parallel mit `nThreads > 0` Arbeitern die seiten-effektfreie Funktion `mapFunc` auf jedes Element des Eingabefelds `input` anwenden. Zur Last-verteilung soll dabei ein vereinfachter Arbeitsdiebstahl eingesetzt werden. Jeder Arbeiter kann Aufträge vom nachfolgenden Arbeiter stehlen; der letzte Arbeiter stiehlt vom ersten Arbeiter:



Hinweis: Die Reihenfolge der Ergebnisse in der zurückgegebenen Liste ist irrelevant.

- Erstellen Sie im Feld 1 die `Worker` und bringen Sie diese zur parallelen Ausführung.
- Verteilen Sie im Feld 2 die Eingabe gleichmäßig auf die Warteschlangen der Arbeiter.
- Zum Beenden der Arbeiter soll die Giftpille `POISON_PILL` dienen. Legen Sie daher im mit 3 markierten Feld die Giftpille in die Warteschlange von Arbeiter 0.
- In Feld 4 setzen Sie folgendes Verhalten um: Ist die Warteschlange eines Arbeiters nicht leer, soll er das Kopfelement entnehmen und bearbeiten. Im Fall einer Giftpille gibt der Arbeiter diese an den vorherigen Arbeiter (`prevDeque`) und beendet sich danach. Ist die eigene Warteschlange leer, soll Arbeitsdiebstahl erfolgen. Es ist ein Element vom Ende der Warteschlange des nächsten Arbeiters (`nextDeque`) zu entnehmen. Eine gestohlene Giftpille ist jedoch zurückzugeben.

```
import java.util.concurrent.*;
import java.util.function.*;
import java.util.*;
```

```
public class Mapper<In, Out> {
    final In POISON_PILL = ... // Giftpille

    BlockingQueue<Out> outQueue; // Ausgabe
    Function<In, Out> mapFunc; // seiteneffektfreie Funktion

    public List<Out> map(In[] input, int nThreads) throws Exception {
        this.outQueue = new LinkedBlockingQueue<>();

        List<LinkedBlockingDeque<In>> dequeues = new ArrayList<>();
        for (int i = 0; i < nThreads; ++i) {
            dequeues.add(new LinkedBlockingDeque<>());
        }
    }
}
```

1

2

3

```

List<Out> result = new ArrayList<>();
for (int i = 0; i < input.length; ++i) {
    result.add(this.outQueue.take()); // Ergebnisse einsammeln
}
return result;
} // ENDE VON map

```

```

class Worker extends Thread {
    BlockingDeque<In> inDeque; // Warteschlange dieses Arbeiters
    BlockingDeque<In> prevDeque; // Warteschlange des vorherigen Arbeiters
    BlockingDeque<In> nextDeque; // Warteschlange des naechsten Arbeiters

    Worker(BlockingDeque<In> inDeque, BlockingDeque<In> prevDeque,
           BlockingDeque<In> nextDeque) {
        this.inDeque = inDeque;
        this.prevDeque = prevDeque;
        this.nextDeque = nextDeque;
    }

    public void run() {
        try {
            while (true) {
                In in = inDeque.pollFirst();

```

4

```

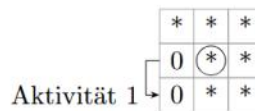
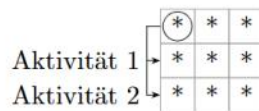
                if (in != null) {
                    outQueue.offer(mapFunc.apply(in)); // Arbeitspaket bearbeiten
                }
            } // ENDE VON while
        } catch (Exception e) { }
    } // ENDE VON run
} // ENDE VON Worker
} // ENDE VON Mapper

```

Aufgabe 5 (Parallele Gauß-Elimination)

(10 Punkte)

Vervollständigen Sie folgende parallele Implementierung des gaußschen Eliminationsverfahrens zur Lösung linearer Gleichungssysteme. Das Verfahren verarbeitet alle Spalten von links nach rechts und versucht, die Einträge unterhalb der Diagonale auf 0 zu setzen. Für eine gegebene Spalte können die Zeilen dabei parallel bearbeitet werden:



In der Phase i werden also in der i -ten Spalte Nullen erzeugt. Dazu sollen parallel alle Zeilen, deren Zeilennummern j größer als i ist, angepasst werden (Methode `makeZero`). Je nach Anzahl verwendeter Aktivitäten muss eine Aktivität auch mehr als eine Zeile bearbeiten.

- Erzeugen Sie in Block 1 zunächst eine Barriere, bevor Sie `nthreads` Aktivitäten (`Worker`) instanziiieren und zur parallelen Ausführung bringen. Die Barriere dient den Aktivitäten später dazu, dass die Elimination erst dann mit der nächsten Spalte fortsetzt, wenn die Bearbeitung für alle Zeilen abgeschlossen ist. Die Methode `eliminate` soll erst dann verlassen werden, wenn die Elimination vollständig abgeschlossen ist.
- Vervollständigen Sie weitere, notwendige Attribute der Aktivitäten in Block 2 und passen Sie den Konstruktor entsprechend an. Da sich die Zeilennummer, für die eine Aktivität zuständig ist, in jeder Phase ändert, ist eine zyklische Aufteilung vorzunehmen. Sie sollen also keine Zeilenbereichsgrenze als Attribute der Aktivitäten vorsehen.
- Ergänzen Sie das Eliminationsverfahren in Block 3. Verteilen Sie dazu die in Phase i verbleibenden Zeilen der Matrix zyklisch auf die Aktivitäten. Sorgen Sie ferner dafür, dass keine Aktivität den Phasenwechsel vollzieht, ehe alle Zeilen bearbeitet wurden.

```
import java.util.*;
import java.util.concurrent.*;
public class Gaussian {

    double[][] matrix;          // Gleichungssystem
    final int nThreads = ...    // Anzahl zu verwendender Aktivitaeten

    public void eliminate() throws Exception {
```

1

```

    }

```

```

class Worker extends Thread {

```

```

    int id;

```

```

    Worker(int id,                                ) {

```

```

        this.id = id;

```

```

    }

```

```

    @Override

```

```

    public void run() {

```

```

        try {

```

```

            for (int i = 0; i < matrix[0].length; ++i) {

```

```

                for (int j =                                ) {

```

```

                    makeZero(i, j); // Eliminiere (i,j) mithilfe matrix[i][i]

```

```

                }

```

```

            }

```

```

        } catch (Exception e) { /* Ignore */ }

```

```

    }

```

```

} // ENDE class Worker

```

```

}

```

2

3

Aufgabe 4 (ExecutorService)

(13 Punkte)

Vervollständigen Sie die unten gegebene Implementierung eines `ExecutorService`. Dieser soll insgesamt `numThreads` Arbeiter (Klasse `Worker`) verwalten, denen er Arbeit in Form von `FutureTask<?>`-Objekten über eine Warteschlange übergibt. Die `Worker` sollen sich beenden, sobald sie eine `POISON_PILL` erhalten. Diese werden verschickt, wenn der `ExecutorService` in einen Zustand umgeschaltet wird, in dem er keine weiteren Arbeitspakete mehr annimmt. Die boolesche Variable `shutdown` soll diesen Zustand repräsentieren. Achten Sie darauf, dass die Methoden des `ExecutorService` thread-sicher sind.

- Deklarieren Sie eine blockierende Warteschlange `input` und weitere Attribute in Feld 1.
- Sorgen Sie in Feld 2 dafür, dass `numThreads` `Worker` erzeugt und zur parallelen Ausführung gebracht werden. Die notwendige Schleife ist bereits gegeben.
- Ergänzen Sie Feld 3, so dass ein `Worker` ein Arbeitspaket entnimmt oder von der Warteschlange blockiert wird. Achten Sie auf die richtige Behandlung einer `POISON_PILL`.
- Die Methode `shutdown` in Feld 4 soll dafür sorgen, dass keine weiteren Arbeitspakete angenommen werden. Ferner sollen sich nach einem Aufruf von `shutdown` die `Worker` beenden, sobald es keine weiteren Arbeitspakete gibt.
- Übergeben Sie in Feld 5 das Arbeitspaket `task` zur Ausführung an die `Worker`, sofern `shutdown` noch nicht aufgerufen wurde. Andernfalls soll mittels Aufruf von `reject` eine entsprechende Ausnahme geworfen werden. Stellen Sie sicher, dass ein zurückgegebenes Arbeitspaket auch tatsächlich ausgeführt wird, und vermeiden Sie Wettlaufsituationen zwischen `shutdown` und `submit`.

```
import java.util.*;
import java.util.concurrent.*;
```

```
public class ExecutorService {
    int numThreads;
```

input
= new

1

```
static final FutureTask<?> POISON_PILL = // ...
```

```
public ExecutorService(int numThreads) {
    this.numThreads = numThreads;
    for (int i = 0; i < numThreads; ++i) {
```

2

```
    }
}
```

```
class Worker extends Thread {  
    public void run() {  
        try {  
            FutureTask<?> task;
```

```
            while (
```

```
) {
```

3

```
                task.run();  
            } // ENDE while  
        } catch (Exception e) { }  
    }  
} // ENDE class Worker
```

```
public void shutdown() {
```

4

```
}
```

```
public Future<?> submit(Callable<?> callable) {
```

5

```
    FutureTask<?> task = new FutureTask<>(callable);
```

```
    return task;
```

```
}
```

```
private void reject() {  
    throw new RejectedExecutionException();  
}  
} // ENDE class ExecutorService
```

Aufgabe 3 (MapReduce)

(14 Punkte)

Ergänzen Sie in dieser Aufgabe die gegebene parallele MapReduce-Implementierung. Gemäß dem MapReduce-Schema soll die Methode `execute` die seiteneffektfreien Funktionen (gegeben durch `mapper` und `reducer`) auf die in `inputs` übergebenen Daten anwenden. Achten Sie darauf, dass Ihre Umsetzung thread-sicher ist. Ändern Sie keinen vorgedruckten Code.

- Erstellen Sie in Feld 1 einen `ExecutorService`, der stets mit genau `nThreads` Aktivitäten arbeitet.
- Bringen Sie in Feld 2 die Arbeitspakete aus `callables` zur parallelen Ausführung und sammeln Sie deren Ergebnisse in der Liste `result`.
- Legen Sie in Feld 3 die Ergebnisse der `map`-Funktion so in die globale Tabelle `comb`, dass diese nach Ausführung aller `map`-Arbeitspakete jedem Schlüssel alle von `map` erzeugten Werte zuordnet.

```
import java.util.*;
import java.util.concurrent.*;
public class ParallelMapReduce<Key, Val> {
    class Pair {
        ...
        Key getKey() { ... }
        Val getValue() { ... }
    }
    abstract class Mapper {
        abstract List<Pair> map(Key key, Val value);
    }
    abstract class Reducer {
        abstract Pair reduce(Key key, List<Val> values);
    }
    Mapper mapper; // bei Initialisierung gesetzt
    Reducer reducer; // bei Initialisierung gesetzt
    int nThreads; // Anz. Aktivitaeten - bei Initialisierung gesetzt

    public List<Pair> execute(List<Pair> inputs) throws Exception {
        Map<Key, List<Val>> comb = new HashMap<>();

        ExecutorService exServ =

    try {
        // Erstelle map-Arbeitspakete und fuehre sie aus
        List<Callable<Pair>> mapTasks = new ArrayList<>();
        for (Pair p : inputs) {
            mapTasks.add(new MapTask(p, comb));
        }
        executeAndWait(exServ, mapTasks);

        // Erstelle reduce-Arbeitspakete und fuehre sie aus
        List<Callable<Pair>> reduceTasks = new ArrayList<>();
        for (Key k : comb.keySet()) {
            reduceTasks.add(new ReduceTask(k, comb.get(k)));
        }
        return executeAndWait(exServ, reduceTasks);
    } finally { exServ.shutdown(); }
}
```

1

```
List<Pair> executeAndAwait(ExecutorService exServ,
    List<Callable<Pair>> callables) throws Exception {
```

```
List<Pair> result = new ArrayList<>();
List<Future<Pair>> futures = new ArrayList<>();
```

2

```
    return result;
}
```

```
class MapTask implements Callable<Pair> {
    Pair pair;
    Map<Key, List<Val>> comb;
    MapTask(Pair pair, Map<Key, List<Val>> comb) {
        this.pair = pair;
        this.comb = comb;
    }
    public Pair call() {
        for (Pair p : mapper.map(pair.getKey(), pair.getValue())) {
```

3

```
        }
        return null;
    }
} // END of class MapTask
```

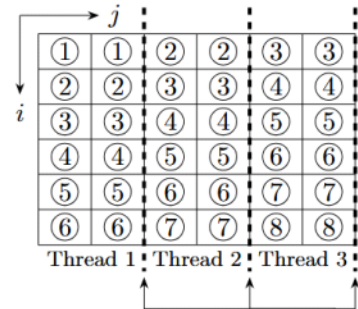
```
class ReduceTask implements Callable<Pair> { ... }
} // END of class ParallelMapReduce
```

Aufgabe 3 (Fließband)

(11 Punkte)

Setzen Sie in dieser Aufgabe eine Fließband-Parallelisierung eines auf dynamischer Programmierung basierenden Algorithmus um. Zwischenergebnisse werden im zweidimensionalen Feld `matrix` gespeichert. Um den Wert eines Eintrags (i, j) mittels `computeValue` zu berechnen, müssen die Werte $(i, j - 1)$ (links) sowie $(i - 1, j)$ (darüber) bereits bekannt sein. Falls $(i, j - 1)$ oder $(i - 1, j)$ außerhalb von `matrix` liegt, nimmt `computeValue` einen Standardwert an.

Um die Berechnung der Einträge von `matrix` zu parallelisieren, sollen die Spalten von `matrix` auf die verwendeten Threads aufgeteilt werden (siehe Abbildung rechts). Hat ein Thread seinen Anteil an einer Zeile vollständig berechnet, soll er dies dem nachfolgenden Thread durch Weitergabe des Objekts `XCH` über eine `BlockingQueue` mitteilen. Mit Ausnahme des ersten Threads besitzt somit jeder Thread eine Eingabe- (`in`) und eine Ausgabe-Warteschlange (`out`). Die sich so ergebenden Berechnungsphasen sind rechts in der Matrix dargestellt (z. B. ①).



- Instanzieren Sie in den mit 1 markierten Feldern die notwendigen Warteschlangen (•), teilen Sie die Spalten der Matrix annähernd gleichmäßig auf `nThreads` Threads auf und bringen Sie diese zur parallelen Ausführung. Dabei soll für jeden Thread eine Ausgabe-warteschlange (`out`) erzeugt werden, die dann als Eingabewarteschlange (`in`) des nächsten Threads fungiert. Zuletzt soll die letzte Warteschlange in der Variablen `out` stehen.
- Setzen Sie im Feld 2 das Fließband-Muster mithilfe von Warteschlangen um. Ein Thread soll nur dann seinen Teil einer Zeile bearbeiten, wenn die dazu notwendigen Werte bereits eingetragen wurden.

```
import java.util.concurrent.*;

public class Pipeline {
    static final Object XCH = new Object();

    static int[][] matrix;

    // Berechnet Eintrag (i, j) in matrix
    private void computeValue(int i, int j) { /* ... */ }

    public void compute(final int nThreads) throws Exception {
        int nRows = matrix.length;
        int nCols = matrix[0].length;
        BlockingQueue<Object> in = null;
        BlockingQueue<Object> out = null;
    }
}
```

1

```
for (int i = 0; i < nThreads; ++i) {
```

1

```

    }
    for (int i = 0; i < nRows; ++i) {
        out.take(); // Wait for completion
    }
} // END OF METHOD compute

class Stage extends Thread {
    int from, to; // Columns to compute
    BlockingQueue<Object> in, out;

    Stage(int from, int to, BlockingQueue<Object> in,
        BlockingQueue<Object> out) {
        this.from = from;
        this.to = to;
        this.in = in;
        this.out = out;
    }

    public void run() {
        try {
            for (int i = 0; i < matrix.length; ++i) {
```

2

```

                for (int j =          ; j <          ; ++j) {

                    computeValue(i, j);
                }
            }
        } catch (Exception e) { }
    } // END OF METHOD run
} // END OF CLASS Stage
} // END OF CLASS Pipeline
```

Aufgabe 4 (Nebenläufige Halde)

(12 Punkte)

In dieser Aufgabe ist eine Implementierung einer binären Halde gegeben, deren Wurzel das minimale Element enthalten soll. Sorgen Sie dafür, dass der Halde **nebenläufig** Elemente hinzugefügt werden können, so dass nach Abschluss aller Einfügeoperationen die Haldeneigenschaft gilt.

Die Elemente der Halde liegen in der Reihung `heapArray`. Sie können davon ausgehen, dass diese immer groß genug ist, um alle Elemente aufnehmen zu können. Jeder Position in der Halde wird über `lockArray` ein `ReentrantLock` zugeordnet. Das Einfügen in der Methode `add` fügt ein neues Element zunächst als Blatt hinzu, bevor dieses dann solange mit seinem jeweiligen Eltern-Knoten vertauscht wird, bis es am richtigen Platz steht.

Ihre Lösung soll nebenläufiges Einfügen ermöglichen, ohne dass eine vollständige Serialisierung der `add`-Methode erzwungen wird. Sperren Sie daher die Knoten so, dass während jedes Schritts des Vertauschens höchstens der aktuelle Knoten und dessen Eltern-Knoten gesperrt sind. Befindet sich ein Element an der richtigen Position, soll die zugehörige Sperre wieder freigegeben werden, bis schlussendlich alle Sperren freigegeben sind. Vermeiden Sie Wettlaufsituationen bei mehreren nebenläufigen Einfügeoperationen (z.B. kann es jederzeit nach Vergabe eines Index passieren, dass ein Kind-Knoten des aktuellen Knotens beschrieben wird).

Hinweis: Sie können davon ausgehen, dass nur die `add`-Methode nebenläufig aufgerufen wird und dass während der Ausführung keine Ausnahme auftritt.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.locks.*;
import java.util.concurrent.atomic.*;

public class ConcurrentHeap {
    Object[] heapArray;
    ReentrantLock[] lockArray;
    int size; // number of items in Heap

    ConcurrentHeap(int maxSize) {
        this.heapArray = new Object[maxSize];
        this.lockArray = new ReentrantLock[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            this.lockArray[i] = new ReentrantLock();
        }
    }

    int parent(int index) { return (index - 1) / 2; }

    // Compares heapArray[firstIdx] and heapArray[secondIdx]:
    boolean isSmallerThan(int firstIdx, int secondIdx) { /* ... */ }

    void swap(int i1, int i2) {
        final Object tmp = this.heapArray[i1];
        this.heapArray[i1] = this.heapArray[i2];
        this.heapArray[i2] = tmp;
    }
}
```

```
void add(final Object object) throws Exception {
    int index;

    index = this.size++;

    this.heapArray[index] = object;

    // ===== Vertausche =====
    int current = index;
    boolean anotherIteration = true;

    while (current > 0 && anotherIteration) {

        boolean needsSwap = isSmallerThan(current, parent(current));
        anotherIteration = needsSwap;

        if (needsSwap) {
            swap(current, parent(current));
        }

        current = parent(current);

    }

} // END OF METHOD add
} // END OF CLASS ConcurrentHeap
```

Aufgabe 2 (Parallele Dateisuche)

(15 Punkte)

In dieser Aufgabe entwickeln Sie eine einfache parallele Dateisuche, die eine Verzeichnisstruktur traversiert und jede Datei verarbeitet. Dazu erhält sie über die Methode `beginSearch` ein Startverzeichnis sowie die Anzahl der maximal gleichzeitig aktiven `SearchThreads`.

Das Ausführungsmodell sei wie folgt: Die Dateisuche beginnt mit einem Thread und aktiviert nach und nach weitere `SearchThreads`, bis eine Obergrenze an aktiven Threads erreicht ist.

Die Dateisuche ist konzeptuell in zwei Teile unterteilt. Die Verwaltung der Threads übernimmt die Methode `startThread`, während die Dateisuche in der Methode `search` geschieht. Sowohl der Haupt-Thread als auch die `SearchThreads` rufen dabei `startThread` auf.

Hinweis: Gehen Sie davon aus, dass die Methode `beginSearch` nicht parallel aufgerufen wird.

- a) Die Anzahl der Threads soll durch Zählen der aktiven `SearchThreads` begrenzt werden. Deklarieren und initialisieren Sie im Feld 1 die erforderlichen Datenstrukturen.
- b) Die Methode `startThread` im Feld 2 soll entscheiden, ob für ihr Argument `file` ein neuer Thread erzeugt werden darf, ohne die maximale Anzahl (`maxThreads`) zu überschreiten. Ist dies der Fall, soll `true` zurückgegeben werden, andernfalls `false`. Formulieren Sie eine geeignete Bedingung für die `if`-Anweisung. Wird diese zu `true` ausgewertet, erzeugen Sie einen neuen `SearchThread` und bringen diesen zur parallelen Ausführung.
- c) Modifizieren Sie, falls erforderlich, in der Methode `run` im Feld 3 Ihren Zähler der aktiven Threads geeignet.
- d) Durch Verknüpfungen (Links) kann es passieren, dass die Dateisuche die selbe Datei mehrfach erreicht. Um jede Datei nur einmal zu verarbeiten, sollen gefundene Dateien in der Menge `foundFiles` abgelegt werden. Enthält `foundFiles` das aktuelle `childFile` noch nicht (Methode `contains`), soll dieses hinzugefügt werden (Methode `add`) und `doVisit` auf `true` gesetzt werden. Ergänzen Sie entsprechenden, thread-sicheren Code im Feld 4.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
```

```
class FileSearch {
    int maxThreads;
```

```
        threadCount =
```

1

```
        Set<File> foundFiles = new HashSet<File>(); // Gefundene Dateien
```

```
    void process(File file) { /* ... */ } // Verarbeitet file
```

```
    public void beginSearch(File startFile, int maxThreads)
        throws Exception {
        this.maxThreads = maxThreads;
        foundFiles.add(startFile);
        startThread(startFile);
    } // END of beginSearch
```

```
    boolean startThread(File file) {
```

```
        if (threadCount.                                ) {
```

```
            return true;
        } else {
```

```
            return false;
        }
```

2

```
    }
```

```

class SearchThread extends Thread { // innere Klasse
    private final Queue<File> fileQueue = new ArrayDeque<>();

    public SearchThread(File startFile) {
        this.fileQueue.add(startFile);
    }

    public void run() {

```

```

        search();

```

```

    }

```

```

    void search() {
        while (!fileQueue.isEmpty()) {
            final File file = fileQueue.remove();

            if (file.isDirectory()) {
                for (File f : file.listFiles()) {
                    final File childFile = f.getCanonicalFile();
                    boolean doVisit = false;

```

```

                        if (doVisit) {
                            if (!startThread(childFile)) {
                                fileQueue.add(childFile);
                            }
                        }
                    } // END of for loop
                } else {
                    process(file); // Verarbeitet file
                }
            } // END of while loop
        } // END of search()
    } // END of SearchThread
} // END of FileSearch

```

Aufgabe 3 (Turnierverfahren)

(13 Punkte)

Setzen Sie ein Turnierverfahren zur parallelen Verschmelzung mehrerer Listen um (analog zum in der Vorlesung gezeigten Schema). Dazu erhält die Methode `mergeLists` eine Liste `lists` zu verschmelzender Listen, die parallel von `numThreads` Aktivitäten bearbeitet werden sollen. Sie können davon ausgehen, dass `lists` immer mindestens zwei Listen enthält. Die Verschmelzung zweier Listen übernimmt die vorgegebene, assoziative Methode `merge`.

Nutzen Sie das Feld 1, um notwendige Instanzvariablen zu deklarieren und zu initialisieren.

- a) Berechnen Sie in Feld 2 abhängig von der Schrittweite `d` die Positionen (Indices in `lists`) der beiden zu verschmelzenden Listen. Beachten Sie, dass die verschmolzene Liste am Index `first` abgelegt wird und dass die finale Ergebnisliste an Index 0 erwartet wird.
- b) Synchronisieren Sie das Turnierverfahren in Feld 3 geeignet und aktualisieren Sie die Schrittweite `d`.
- c) Erstellen Sie in Feld 4 alle für die Synchronisation notwendigen Objekte. Bringen Sie die Aktivitäten zur parallelen Ausführung und warten Sie auf deren Beendigung.

```
import java.util.*;
import java.util.concurrent.*;
public class Turnier<E> {

    List<List<E>> lists;

    private List<E> merge(List<E> list1, List<E> list2) { /* ... */ }

    class Worker extends Thread {

        int index;

        Worker(int index

        ) {

        }

    }

}
```

1

```
public void run() {  
    int d = 1; // Schrittweite  
    try {  
        while (d < lists.size()) {
```

```
            int first =  
  
            int second =
```

2

```
            if (second < lists.size()) {  
                lists.set(first,  
                    merge(lists.get(first), lists.get(second)));  
            }
```

3

```
        }  
    } catch (Exception e) { /* Ignore */ }  
} // END OF METHOD run  
} // END OF CLASS Worker
```

```
List<E> mergeLists(List<List<E>> lists) throws Exception {  
    this.lists = lists;  
  
    int numThreads = lists.size() / 2;
```

4

```
    Worker[] threads = new Worker[numThreads];  
  
    for (int i = 0;
```

```
        return lists.get(0);  
    }  
}
```

Aufgabe 2 (Monte-Carlo)

(13 Punkte)

Die Kreiszahl π soll mittels Monte-Carlo-Simulation parallel wie folgt berechnet werden. Die bereits vorgegebene Funktion `sample` liefert mit einer Wahrscheinlichkeit von $\frac{\pi}{4}$ das Ergebnis `true` zurück. π lässt sich also nähern, indem `sample` parallel mehrfach aufgerufen wird und die Ergebnisse mitgezählt werden.

Tipp: Als Beispiel für die Verwendung der zu vervollständigenden Klasse können Sie die ganz unten abgedruckte `main`-Methode heranziehen.

- a) Ergänzen Sie in Feld 1 die Methode `updateCount`, die die Zählvariablen `passes` und `hits` um die Argumentwerte inkrementiert. Vervollständigen Sie die Methode `getPi`, die die aktuelle Näherung von π zurückliefert. Achten Sie auf ausreichende Synchronisation zwischen beiden Methoden, da diese nebenläufig aufgerufen werden können.
- b) Geben Sie in Feld 2 die Methoden `startComputation` und `waitForComputation` an. Erstere soll die gegebene Anzahl an Worker-Arbeitsfäden starten, um `samples` Simulationen durchzuführen. Achten Sie darauf, die Last möglichst gleichmäßig aufzuteilen. Stellen Sie sicher, dass jeder Worker einen anderen (ansonsten beliebigen) `seed` erhält. `waitForComputation` soll auf die Terminierung aller Worker warten.
- c) Vervollständigen Sie in Feld 3 die Klasse `Worker`. Sobald ein solcher Thread gestartet wird, soll er die ihm zugeteilte Anzahl an Simulationen mittels `sample` durchführen. Damit schon erste Abschätzungen möglich sind, während die Berechnungen noch laufen, soll jeder Thread alle 1000 Durchläufe sowie nach Abschluss der letzten Simulation das Ergebnis über `updateCount` im `PiComputer` abgeben.

```
import java.util.Random;
public class PiComputer {
```

```
    private long passes; // Durchgeführte Simulationen
    private long hits;    // Erhaltene trues
    private void updateCount (long newPasses, long newHits) {

    }

    public double getPi() {

    }

}
```

1

```
private Worker[] workers;
```

```
public void startComputation(long samples, int threads) {  
    workers =
```

```
    for (int i = 0; i < threads; i++) {
```

```
    }
```

```
public void waitForComputation() throws Exception {
```

```
}
```

2

```

class Worker extends Thread {
    private final Random generator;

    private boolean sample() {
        double x = generator.nextDouble();
        double y = generator.nextDouble();
        return Math.sqrt(x*x+y*y) < 1.0;
    }

```

```

    private long load;

```

```

    public Worker(
                                , int seed) {

```

```

        generator = new Random(seed);
    }

```

```

    public void run() {

```

```

        for (long i = 0;
                                ){

```

```

    }

```

```

public static void main(String[] args) {
    PiComputer p = new PiComputer();
    p.startComputation(10000000000L, 16);
    try {
        Thread.sleep(500);
        System.out.println(p.getPi());
        Thread.sleep(500);
        System.out.println(p.getPi());
        p.waitForComputation();
        System.out.println(p.getPi());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

3

Aufgabe 3 (Mandelbrot)

(10 Punkte)

Die vorgegebene Methode `check` gibt für die ihr übergebenen Werte zurück, ob sie Teil der Mandelbrotmenge sind. Die Methode soll für jeden Index im 2D-Array `field` mittels dynamischer Lastverteilung parallel ausgewertet werden.

```
boolean check(int x, int y);
```

- a) Setzen Sie in Feld 1 alle erforderlichen Attribute und starten Sie die `Worker`.
- b) Jede Spaltennummer (erste Dimension) des Arrays entspricht einem Arbeitspaket. Befüllen Sie in Feld 2 die vorgegebene `BlockingQueue` entsprechend (`add`). Denken Sie daran, den Arbeiter-Threads mit Hilfe der Warteschlange zu signalisieren, wann sie sich beenden dürfen. Warten Sie anschließend auf deren Beendigung.
- c) Vervollständigen Sie in Feld 3 die Klasse `Worker`. Entnehmen Sie Arbeitspakete / Spaltennummern (`take`) und rufen Sie `check` für alle Indices der entsprechenden Spalte auf. Die Ergebnisse sollen in `field` geschrieben werden. Sorgen Sie dafür, dass garantiert alle Arbeitspakete abgearbeitet werden und sich die `Worker` anschließend beenden.

```
import java.util.concurrent.*;
```

```
public class Mandelbrot {  
    private final int POISON_PILL = -42;  
    private final BlockingQueue<Integer> queue =  
        new LinkedBlockingDeque<Integer>();  
    public boolean [][] field;  
    Worker[] workers;  
  
    public void compute (boolean [][] field, int nThreads){
```

1

2

```
} // END COMPUTE
```

```
class Worker extends Thread {
```

```
    public void run() {
```

```
        int column;
```

```
        try {
```

```
            while (
```

```
        } catch (Exception e) { }
```

```
    } // END RUN
```

```
    boolean check(int x, int y) {...}
```

```
}
```

```
}
```

3

Aufgabe 4 (Rucksack)

(12 Punkte)

Es soll das Rucksackproblem parallel gelöst werden. Gegeben seien n Objekte mit den Größen $(s_0, s_1, \dots, s_{n-1})$ und den Werten $(v_0, v_1, \dots, v_{n-1})$. Für eine Größe m ist eine Teilmenge der Objekte gesucht, welche den Gesamtwert maximiert, ohne dabei die Gesamtgröße m zu überschreiten ($n, m, s_i, v_i \in \mathbb{N}$). Um den maximalen Gesamtwert $k(n, m)$ effizient zu berechnen, sollen die $k(i, j)$ in eine Tabelle eingetragen werden, deren Zeilen (i) für die Objekte und deren Spalten (j) für den verfügbaren Platz

$$k(i, j) = \begin{cases} 0 & i = 0 \wedge j < s_0 \\ v_0 & i = 0 \wedge j \geq s_0 \\ k(i-1, j) & j < s_i \\ \max(k(i-1, j), k(i-1, j-s_i) + v_i) & \text{sonst} \end{cases}$$

stehen. Alle Einträge innerhalb einer Zeile sollen vom selben Thread bearbeitet werden. Um den Grad der Parallelität zu maximieren, sollen die Zeilen so einzeln zyklisch auf die gegebene Anzahl Worker verteilt werden, dass eine möglichst gleichmäßige Lastverteilung entsteht. Mit einem Array an BlockingQueues stellen Sie sicher, dass keine noch nicht berechneten Werte gelesen werden.

- Setzen Sie in Feld 1 alle erforderlichen Attribute und starten Sie die Worker. Verwenden Sie genau eine LinkedBlockingQueue pro Worker.
- Befüllen Sie nun in Feld 2 die Tabelle k . Ein Thread signalisiert in der eigenen Queue die Fertigstellung jedes Eintrags einzeln, damit der Thread, der die Folgezeile bearbeitet, nicht voraussetzt.

```
import java.util.concurrent.*;
public class Knapsack {
    private static final Object READY = new Object();
    BlockingQueue<Object>[] signals;
    Worker[] workers;
    int[][] k;
    int[] sizes, values;

    int solve(int[] sizes, int[] values, int space, int numThreads)
        throws InterruptedException {
```

```
        k = new int[sizes.length][
        signals = new BlockingQueue[
        workers = new Worker[
        this.sizes = sizes; this.values = values;
```

1

```

    for(int i = 0; i < numThreads; i++) {
        workers[i].join();
    }
    return k[sizes.length-1][space];
} // END SOLVE

class Worker extends Thread {
    int id, numThreads;

    public Worker(int id, int numThreads) {
        this.id = id;
        this.numThreads = numThreads;
    }

    public void run() {
        try {
            // Zyklische Aufteilung:
            for (int i = 0; i < numThreads; i++) {

                // Einträge einer Zeile:
                for (int j = 0; j < sizes[i].length; j++) {

                    if( j % numThreads == i ) { // Sonderfall 1. Objekt

                        // ...

                    } else {

                        // ...

                    }

                }

                // Fertigstellung signalisieren:
                finished.add(i);

            }

        } catch (InterruptedException e) {}

    } // END RUN
} // END WORKER
} // END KNAPSACK

```

2

Aufgabe 3 (Crypto-Mining)

(13 Punkte)

Beim Bitcoin-Mining wird systematisch nach Werten gesucht, deren sha256-Hash kleiner oder gleich einem bestimmten Schwellenwert ist. Eine vereinfachte parallele Version soll in dieser Aufgabe implementiert werden. Hierfür soll der zu untersuchende Bereich `[from, to[` möglichst gleichmäßig auf `nThreads` Miner aufgeteilt werden, die ihren Teilbereich nach gültigen Werten durchsuchen.

- Vervollständigen Sie in Feld 1 die Klasse `Miner`. Setzen Sie im Konstruktor alle erforderlichen Attribute. Die Funktion `call` soll eine `LinkedList` aller gefunden gültigen Werte zurückliefern. Ein Wert ist gültig, wenn sein sha256-Hash weder negativ noch größer als der Schwellenwert `threshold` ist.
- Erstellen Sie nun in Feld 2 `nThreads` Miner. Teilen Sie dabei den abzusuchenden Wertebereich `[from, to[` möglichst gleichmäßig auf. Sie dürfen dabei annehmen, dass es weniger Threads als abzusuchende Werte gibt. Nutzen Sie einen `ExecutorService`, um die Miner auf `nThreads` Threads parallel zur Ausführung zu bringen.
- Warten Sie nun in Feld 3 auf die Beendigung der Miner. Anschließend sollen die Ergebnisse aller Miner zu einer Gesamtliste zusammengefügt und zurückgegeben werden.

Tipp: Eine List kann mit der Methode `addAll` an eine andere List angehängt werden.

```
import java.util.*;
import java.util.concurrent.*;
```

```
public class CryptoMining {
    long threshold;

    public static long sha256(long in){ ... }

    class Miner implements Callable<List<Long>> {
```

```
        long mineFrom, mineTo;

        public Miner(
            ...
        ){

        }

        public LinkedList<Long> call() throws Exception {
            LinkedList<Long> l =

            return l;
        }
    } // END MINER
```

1

```
public LinkedList<Long> mine(  
    long from, long to, long threshold, int nThreads) {  
    this.threshold = threshold;  
  
    long blockSize =  
  
    LinkedList<Future<List<Long>>> futs =  
  
    ExecutorService e =
```

2

```
        e.shutdown();
```

```
        LinkedList<Long> res =
```

```
        return res;  
    }
```

3

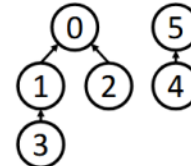
```
} // END CRYPTOMINING
```

Aufgabe 3 (Zeigerverdopplung)

(16 Punkte)

Gegeben sei ein `int`-Array `parents`, in dem ein oder mehrere Bäume gespeichert sind. Dazu ist in jedem Element der Index des direkten Elternknotens gespeichert bzw. bei Wurzelknoten `-1`. Mit Hilfe der Zeigerverdopplung soll der Abstand `depths[i]` jedes Knotens `i` von der jeweiligen Wurzel parallel ermittelt werden.

	0	1	2	3	4	5
<code>parents</code>	-1	0	0	1	5	-1
<code>depths</code>	0	1	1	2	1	0



- a) Vervollständigen Sie in Feld 1 die Methode `findDepths`. Starten Sie dafür `nThreads` Worker, die jeweils die Knoten von Index `from` (inklusive) bis `to` (exklusiv) bearbeiten. Achten Sie auf eine gleichmäßige Aufteilung der Arbeitspakete. Sie dürfen annehmen, dass es mehr Array-Elemente als Threads gibt. Warten Sie anschließend auf die Beendigung aller Worker.

```
import java.util.concurrent.CyclicBarrier;
```

```
public class TreeDoubling {
    int[] parentsOld, parentsNew, depths;
    CyclicBarrier barrier;
    Worker[] workers;
    int width;

    int[] findDepths(int[] parents, int nThreads) {
```

```
        assert(nThreads > parents.length);
        parentsOld = parents.clone();
        parentsNew = new int[parents.length];
        depths = new int[parents.length];
        barrier = new CyclicBarrier(nThreads);
        workers = new Worker[nThreads];
        width = 1;
```

1

```
    }
```

- b) Vervollständigen Sie nun Feld 2. Führen Sie dafür die Zeigerverdopplung auf dem Array `parentsOld` aus und schreiben Sie das Ergebnis jeder Runde nach `parentsNew`. Berechnen Sie dabei auch die Werte von `depths`.
- c) Vervollständigen Sie schließlich Feld 3. Vertauschen Sie dazu am Ende jeder Runde die beiden Arrays. Achten Sie außerdem darauf, den Wert `width` aktuell zu halten, sowie auf korrekte Synchronisation.

```
class Worker extends Thread {
    int from, to;

    public Worker(int from, int to) {
        this.from = from;
        this.to = to;
    }

    public void run() {
```

```
        try {
            while (width < parentsOld.length) {
                // Zeigerverdopplung und Tiefenberechnung
```

2

```
                // Synchronisation, Austausch und width
```

3

```
            }
        } catch (Exception ignored) {}
```

```
    }
} // END TREEDOUBLING
```

Aufgabe 3 (Gauß-Filter)

(15 Punkte)

In dieser Aufgabe soll ein paralleler Gaußfilter mit dynamischer Lastverteilung programmiert werden. Die Methode `filter` filtert dabei das übergebene Bild mithilfe von `nThreads` Arbeitern `rounds` mal. Jede Spalte (erste Dimension) stellt dabei ein Arbeitspaket dar. Sie dürfen davon ausgehen, dass das Bild quadratische Abmessungen hat.

- Vervollständigen Sie in Feld 1 die Klasse `Worker`. Sie wendet den Gaußfilter auf das Bild `src` an und speichert das Ergebnis in `dst`. Verwenden Sie hierfür die vorgegebene Methode `computePixel`, die einen einzelnen Pixel `(i,j)` für das übergebene Bild `img` berechnet und das Ergebnis zurückliefert. Die Warteschlange `todo` beinhaltet die zu bearbeitenden Spaltennummern. Die abgeschlossenen Spaltennummern sollen in `done` gelegt werden. Denken Sie an einen geeigneten Mechanismus zur Beendigung der `Worker`.
- Initialisieren Sie in Feld 2 alle benötigten Klassenvariablen und starten Sie die geforderte Anzahl `Worker`.
- Führen Sie in Feld 3 `rounds` Filterungen durch. Vertauschen Sie nach jeder Filterung die beiden Bilder.
- Beenden Sie in Feld 4 die `Worker` und geben Sie das resultierende Bild zurück.

```
public class GaussianFilter {
    float[][] src, dst;
    BlockingQueue<Integer> todo, done;
    final Integer POISON_PILL = -1;

    class Worker extends Thread {

        private float computePixel(float[][] img, int i, int j){ // ...

        public void run(){
            int height = src[0].length;
            while (true){
                try {


```

1

```
float[][] filter
    (float[][] image, int rounds, int nThreads) throws Exception {

    src = image;
```

```
    dst =
    todo =
    done =
```

2

3

4

```
    } // END FILTER
} // END GAUSSIANFILTER
```